

UNIT - III

PACKAGES

Defining Package:

A **java package** is a group of similar types of classes, interfaces and sub-packages.

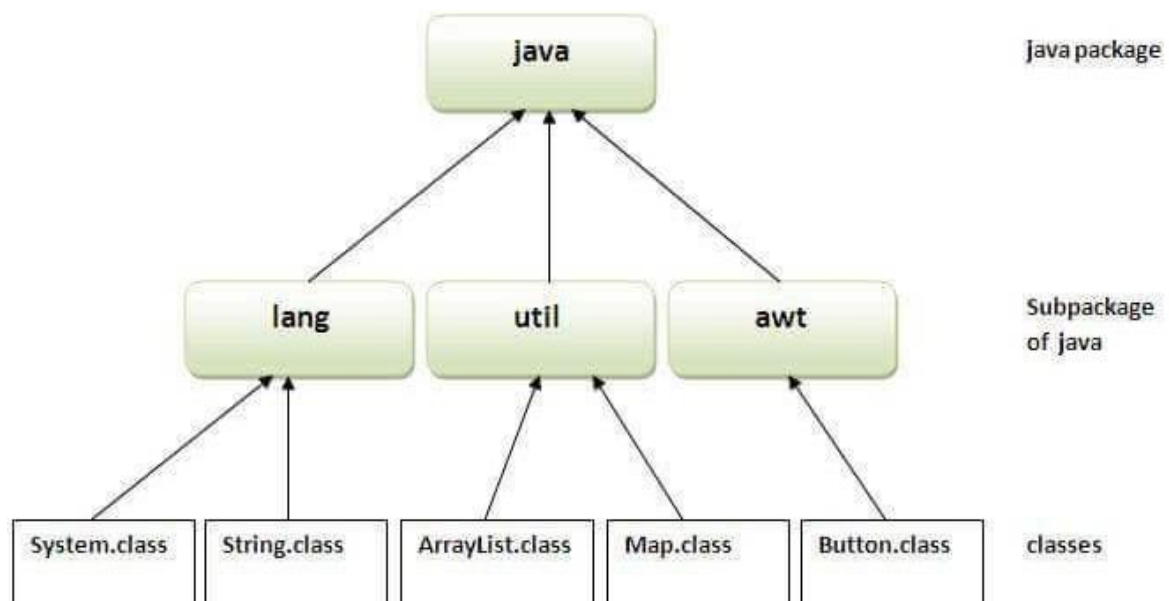
Package in java can be categorized in two form, built-in package and user-defined package.

There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Here, we will have the detailed learning of creating and using user-defined packages.

Advantage of Java Package

- 1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming collision.



Simple example of java package

The **package keyword** is used to create a package in java.

1. //save as Simple.java
2. **package** mypack;
3. **public class** Simple{
4. **public static void** main(String args[]){
5. System.out.println("Welcome to package");
6. }
7. }

How to compile java package

If you are not using any IDE, you need to follow the **syntax** given below:

1. javac -d directory javafilename

For **example**

1. javac -d . Simple.java

The -d switch specifies the destination where to put the generated class file. You can use any directory name like /home (in case of Linux), d:/abc (in case of windows) etc. If you want to keep the package within the same directory, you can use . (dot).

How to run java package program

You need to use fully qualified name e.g. mypack.Simple etc to run the class.

To Compile: javac -d . Simple.java

To Run: java mypack.Simple

Output:Welcome to package

The -d is a switch that tells the compiler where to put the class file i.e. it represents destination.

How to access package from another package?

There are three ways to access the package from outside the package.

1. import package.*;
2. import package.classname;
3. fully qualified name.

1) Using packagename.*

If you use package.* then all the classes and interfaces of this package will be accessible but not subpackages.

The import keyword is used to make the classes and interface of another package accessible to the current package.

Example of package that import the packagename.*

```
1. //save by A.java
2. package pack;
3. public class A{
4.     public void msg(){System.out.println("Hello");}
5. }
```

```
1. //save by B.java
2. package mypack;
3. import pack.*;
4.
5. class B{
6.     public static void main(String args[]){
7.         A obj = new A();
8.         obj.msg();
9.     }
10. }
```

Output:Hello

2) Using packagename.classname

If you import package.classname then only declared class of this package will be accessible.

Example of package by import package.classname

```
1. //save by A.java
2. package pack;
3. public class A
4. {
5.     public void msg()
6.     {
7.         System.out.println("Hello");
8.     }
9. }
1. //save by B.java
2. package mypack;
3. import pack.A;
4.
5. class B{
6.     public static void main(String args[]){
7.         A obj = new A();
8.         obj.msg();
9.     }
10. }
```

Output:Hello

3) Using fully qualified name

If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

It is generally used when two packages have same class name e.g. java.util and java.sql packages contain Date class.

Example of package by import fully qualified name

```
1. //save by A.java
2. package pack;
3. public class A{
4.     public void msg(){System.out.println("Hello");}
5. }

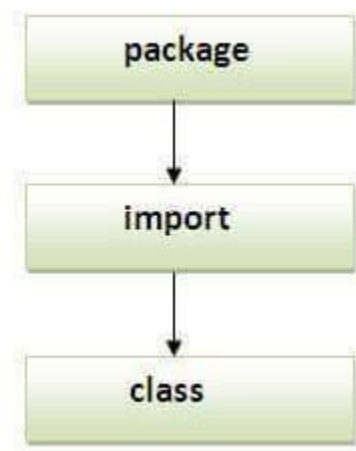
1. //save by B.java
2. package mypack;
3. class B{
4.     public static void main(String args[]){
5.         pack.A obj = new pack.A();//using fully qualified name
6.         obj.msg();
7.     }
8. }
```

Output:Hello

Note: If you import a package, subpackages will not be imported.

If you import a package, all the classes and interface of that package will be imported excluding the classes and interfaces of the subpackages. Hence, you need to import the subpackage as well.

Note: Sequence of the program must be package then import then class.



Subpackage in java

Package inside the package is called the **subpackage**. It should be created **to categorize the package further**.

Let's take an example, Sun Microsystem has defined a package named java that contains many classes like System, String, Reader, Writer, Socket etc. These classes represent a particular group e.g. Reader and Writer classes are for Input/Output operation, Socket and ServerSocket classes are for networking etc and so on. So, Sun has subcategorized the java package into subpackages such as lang, net, io etc. and put the Input/Output related classes in io package, Server and ServerSocket classes in net packages and so on.

The standard of defining package is domain.company.package e.g. com.javatpoint.bean or org.sssit.dao.

Example of Subpackage

1. **package** com.javatpoint.core;
2. **class** Simple{
3. **public static void** main(String args[]){
4. System.out.println("Hello subpackage");
5. }
6. }

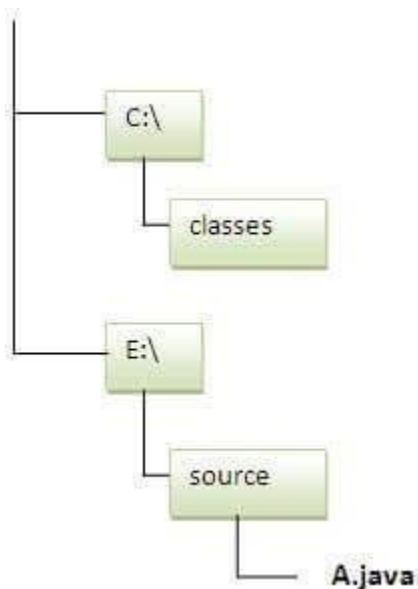
To Compile: javac -d . Simple.java

To Run: java com.javatpoint.core.Simple

Output:Hello subpackage

How to send the class file to another directory or drive?

There is a scenario, I want to put the class file of A.java source file in classes folder of c: drive. For example:



1. //save as Simple.java
2. **package** mypack;
3. **public class** Simple{
4. **public static void** main(String args[]){
5. System.out.println("Welcome to package");
6. }
7. }

To Compile:

```
e:\sources> javac -d c:\classes Simple.java
```

To Run:

To run this program from e:\source directory, you need to set classpath of the directory where the class file

```
e:\sources> set classpath=c:\classes;.;
```

```
e:\sources> java mypack.Simple
```

Another way to run this program by - classpath switch of java:

The -classpath switch can be used with javac and java tool.

To run this program from e:\source directory, you can use -classpath switch of java that tells where to look for class file. For example:

e:\sources> java -classpath c:\classes mypack.Simple

```
Output:Welcome to package
```

Ways to load the class files or jar files

There are two ways to load the class files temporary and permanent.

- Temporary
 - By setting the classpath in the command prompt
 - By -classpath switch
- Permanent
 - By setting the classpath in the environment variables
 - By creating the jar file, that contains all the class files, and copying the jar file in the jre/lib/ext folder.

Rule: There can be only one public class in a java source file and it must be saved by the public class name.

1. //save as C.java otherwise Compile Time Error
- 2.
3. **class A{}**
4. **class B{}**
5. **public class C{}**

How to put two public classes in a package?

If you want to put two public classes in a package, have two java source files containing one public class, name same. For example:

1. //save as A.java
- 2.
3. **package javatpoint;**
4. **public class A{}**

1. //save as B.java
- 2.
3. **package** javatpoint;
4. **public class** B{}

CLASSPATH:

CLASSPATH: CLASSPATH is an environment variable which is used by Application ClassLoader to locate and load the .class files. The CLASSPATH defines the path, to find third-party and user-defined classes that are not extensions or part of Java platform. Include all the directories which contain .class files and JAR files when setting the CLASSPATH.

You need to set the CLASSPATH if:

- You need to load a class that is not present in the current directory or any sub-directories.
- You need to load a class that is not in a location specified by the extensions mechanism.

The CLASSPATH depends on what you are setting the CLASSPATH. The CLASSPATH has a directory name or file name at the end. The following points describe what should be the end of the CLASSPATH.

- If a JAR or zip, the file contains class files, the CLASSPATH end with the name of the zip or JAR file.
- If class files placed in an unnamed package, the CLASSPATH ends with the directory that contains the class files.
- If class files placed in a named package, the CLASSPATH ends with the directory that contains the root package in the full package name, that is the first package in the full package name.

The default value of CLASSPATH is a dot (.). It means the only current directory searched. The default value of CLASSPATH overrides when you set the CLASSPATH variable or using the -classpath command (for short -cp). Put a dot (.) in the new setting if you want to include the current directory in the search path.

If CLASSPATH finds a class file which is present in the current directory, then it will load the class and use it, irrespective of the same name class presents in another directory which is also included in the CLASSPATH.

If you want to set multiple classpaths, then you need to separate each CLASSPATH by a semicolon (;).

The third-party applications (MySQL and Oracle) that use the JVM can modify the CLASSPATH environment variable to include the libraries they use. The classes can be stored in directories or archives files. The classes of the Java platform are stored in rt.jar.

There are two ways to ways to set CLASSPATH: through Command Prompt or by setting Environment Variable.

Let's see how to set CLASSPATH of MySQL database:

Step 1: Click on the Windows button and choose Control Panel. Select System.



Recovery



Region and Language

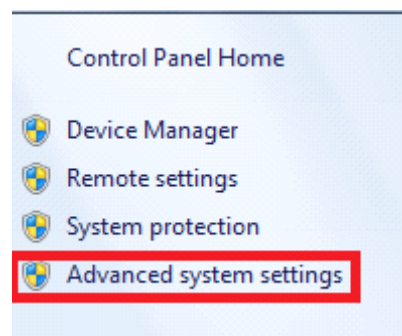


Sync Center

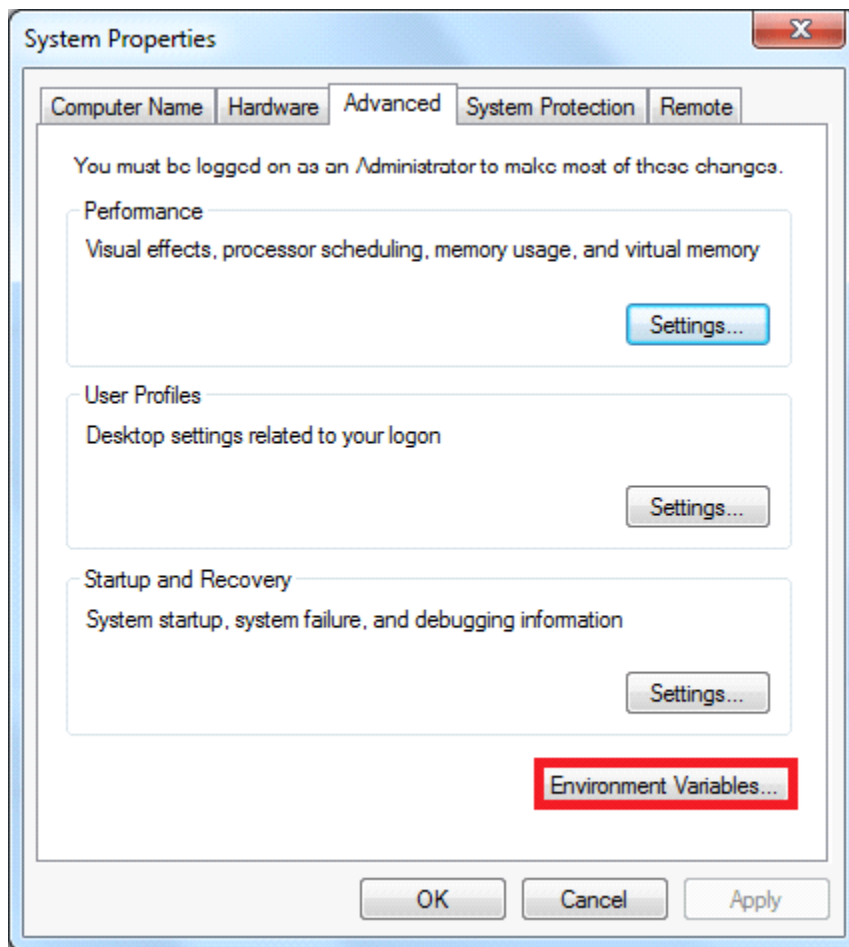


System

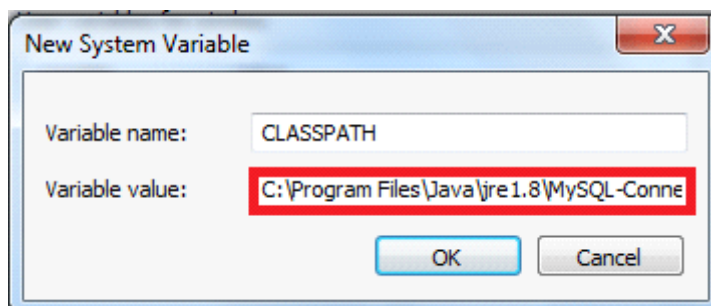
Step 2: Click on **Advanced System Settings**.



Step 3: A dialog box will open. Click on Environment Variables.



Step 4: If the CLASSPATH already exists in System Variables, click on the Edit button then put a semicolon (;) at the end. Paste the Path of MySQL-Connector Java.jar file. If the CLASSPATH doesn't exist in System Variables, then click on the New button and type Variable name as CLASSPATH and Variable value as *C:\Program Files\Java\jre1.8\MySQL-Connector Java.jar;;* Remember: Put ;; at the end of the CLASSPATH.



Difference between PATH and CLASSPATH

PATH	CLASSPATH
PATH is an environment variable.	CLASSPATH is also an environment variable.
It is used by the operating system to find the executable files (.exe).	It is used by Application ClassLoader to locate the .class file.
You are required to include the directory which contains .exe files.	You are required to include all the directories which contain .class and JAR files.
PATH environment variable once set, cannot be overridden.	The CLASSPATH environment variable can be overridden by using the command line option -cp or -CLASSPATH to both javac and java command.

How to Set CLASSPATH in Windows Using Command Prompt

Type the following command in your Command Prompt and press enter.

1. `set CLASSPATH=%CLASSPATH%;C:\Program Files\Java\jre1.8\rt.jar;`

In the above command, The set is an internal DOS command that allows the user to change the variable value. CLASSPATH is a variable name. The variable enclosed in percentage sign (%) is an existing environment variable. The semicolon is a separator, and after the (;) there is the PATH of rt.jar file.

How ext folder works in Java

The ext directory works a bit like the CLASSPATH. ext directory is the part of the class loading mechanism. The classes which are available within JARs in the ext directory are available to Java applications.

The following table demonstrates the key difference between the CLASSPATH and Extension Mechanism:

Characteristics	CLASSPATH	Extension Mechanism
-----------------	-----------	---------------------

Class loading order	CLASSPATH loads after bootstrap and extension loading.	ext loads after bootstrap loading but before CLASSPATH loading.
Scope	It is an application specific. All JREs on the host is the CLASSPATH environment variable.	All JVMs are running in specific JRE java.ext.dirs.
Package name	java.class.path is used to find the directories and JAR archives containing class files.	java.ext.dirs is used to specify where the extension mechanism loads classes.
Specification	It is specified by name including the extension.jar and directory containing .class files.	All JAR files in specified directories are loaded.

The mechanism will pick up all .jar files from the extension directory even if the file does not have the .jar extension. The implementation of this is that if one can change the name of a jar placed in a classpath directory to have an extension other than .jar. The wildcard (*) does not pick it up. This technique will not work with the extension directory.

Let's understand the execution process through an example.

A.java

```

1. public class A
2. {
3.     public String toString()
4.     {
5.         return "hello";
6.     }
7. }
```

B.java

```

1. public class B
2. {
3.     public static void main(final String[] args)
4.     {
```

```
5. System.out.println(new A());  
6. }  
7. }
```

Compile the A.java file. we will archive the compiled A.class file into A.jar. Place this JAR file into another directory than the compiled B.class file.

To demonstrate the use of the classpath, we place the A.jar file in a directory C:\JavaPrograms and will access that JAR through wildcard (*) for B to use.

We found that B can still load the A.class while we had deleted it from the current directory. The Java launcher was explicitly looked for C:\JavaProgram. It is also possible to have the class loaded without its presence in the same directory and explicit classpath specification.

It is often referred to as a benefit of Using the extension mechanism because all applications which are using that JRE can see the same classes without the need to specify them on the classpath explicitly.

What happens if we change the name of A.jar into A.backup in the same CLASSPATH-referenced directory. NoClassDefFoundError is encountered when we do the same because the CLASSPATH-reference does not have the .jar

Using Package Member in Java:

Only public package members are accessible outside the package in which they are defined. To use a public package member from outside its package, you must do one or more of the following:

- Refer to the member by its long (qualified) name
- Import the package member
- Import the member's entire package

Each is appropriate for different situations, as explained in the following sections.

Referring to a Package Member by Name

So far, the examples in this tutorial have referred to types by their simple names, such as `Rectangle` and `StockWatcher`. You can use a package member's simple name if the code you are writing is in the same package as that member or if that member has been imported.

However, if you are trying to use a member from a different package and that package has not been imported, you must use the member's qualified name, which

includes the package name. This is the qualified name for the `Rectangle` class declared in the `graphics` package in the previous example:

```
graphics.Rectangle
```

You could use this long name to create an instance of `graphics.Rectangle`:

```
graphics.Rectangle myRect = new graphics.Rectangle();
```

You'll find that using long names is okay for one-shot uses. But you'd likely get annoyed if you had to write `graphics.Rectangle` again and again. Also, your code would get messy and difficult to read. In such cases, you can just import the member instead.

Importing a Package Member

To import a specific member into the current file, put an `import` statement at the beginning of your file before any class or interface definitions but after the `package` statement, if there is one. Here's how you would import the `Circle` class from the `graphics` package created in the previous section:

```
import graphics.Circle;
```

Now you can refer to the `Circle` class by its simple name:

```
Circle myCircle = new Circle();
```

This approach works well if you use just a few members from the `graphics` package. But if you use many types from a package, you can import the entire package.

Importing an Entire Package

To import all the types contained in a particular package, use the `import` statement with the asterisk (*) wildcard character:

```
import graphics.*;
```

Now you can refer to any class or interface in the `graphics` package by its short name:

```
Circle myCircle = new Circle();
```

```
Rectangle myRectangle = new Rectangle();
```

The asterisk in the `import` statement can be used only to specify all the classes within a package, as shown here. It cannot be used to match a subset of the classes in a package. For example, the following does not match all the classes in the `graphics` package that begin with `A`:

```
import graphics.A*;           // does not work
```

Instead, it generates a compiler error. With the `import` statement, you generally import only a single package member or an entire package.

Note: Another, less common form of import allows you to import only a class and its public inner classes. For example, if the `graphics.Rectangle` class contained useful inner classes, like `Rectangle.DoubleWide` and `Rectangle.Square`, you could import `Rectangle` and its inner classes this way:

```
import graphics.Rectangle.*;
```

For more information, see the section on [Inner Classes](#)◆

For your convenience, the Java compiler automatically imports three entire packages:

- The default package (the package with no name)
- The `java.lang` package
- The current package by default

Note: Packages aren't hierarchical. For example, importing `java.util.*` doesn't let you refer to the `Pattern` class as `regex.Pattern`. You must always refer to it as either `java.util.regex.Pattern` or (if you import `java.util.regex.*`) simply `Pattern`.

Disambiguating a Name

If by some chance a member in one package shares the same name with a member in another package and both packages are imported, you must refer to each member by its qualified name. For example, the previous example defined a class named `Rectangle` in the `graphics` package. The `java.awt` package also contains a `Rectangle` class. If both `graphics` and `java.awt` have been imported, the following is ambiguous:

```
Rectangle rect;
```

In such a situation, you have to be more specific and use the member's qualified name to indicate exactly which `Rectangle` class you want:

```
graphics.Rectangle rect;
```

INTERFACES

Implementing Interface:

An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.

The interface in Java is *a mechanism to achieve [abstraction](#)*. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple [inheritance in Java](#).

In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method body.

Java Interface also **represents the IS-A relationship**.

It cannot be instantiated just like the abstract class.

Since Java 8, we can have **default and static methods** in an interface.

Since Java 9, we can have **private methods** in an interface.

Why use Java interface?

There are mainly three reasons to use interface. They are given below.

- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritance.
- It can be used to achieve loose coupling.



How to declare an interface?

An interface is declared by using the interface keyword. It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default. A class that implements an interface must implement all the methods declared in the interface.

Syntax:

1. **interface** <interface_name>{
- 2.
3. // declare constant fields
4. // declare methods that abstract
5. // by default.
6. }

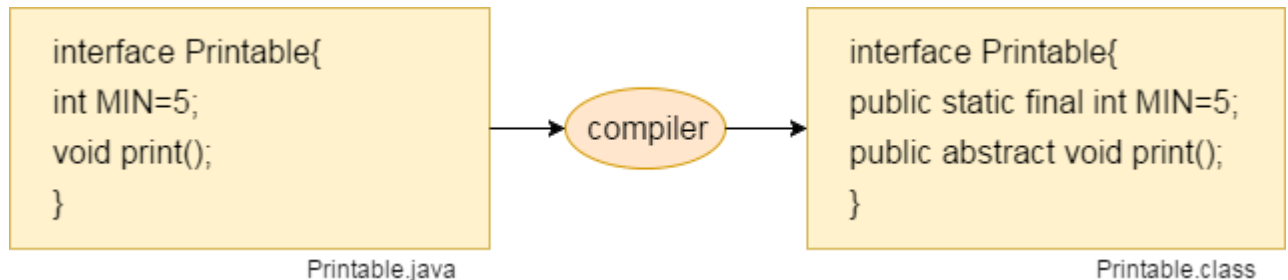
Java 8 Interface Improvement

Since Java 8, interface can have default and static methods which is discussed later.

Internal addition by the compiler

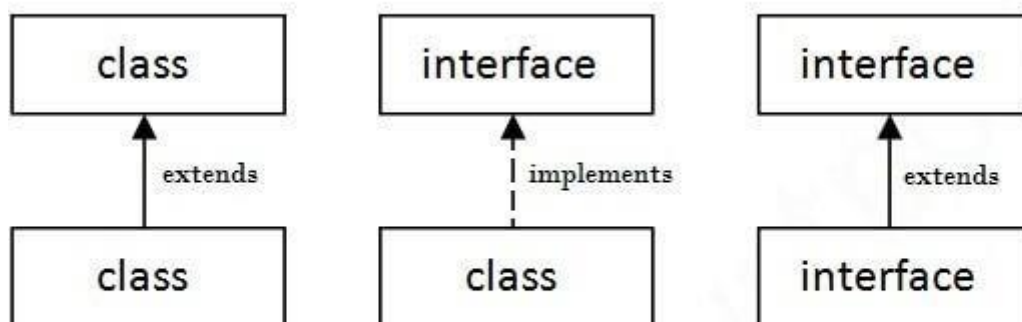
The Java compiler adds *public* and *abstract* keywords before the interface method. Moreover, it adds *public*, *static* and *final* keywords before data members.

In other words, Interface fields are public, static and final by default, and the methods are public and abstract.



The relationship between classes and interfaces

As shown in the figure given below, a class extends another class, an interface extends another interface, but a **class implements an interface**.



Java Interface Example

In this example, the Printable interface has only one method, and its implementation is provided in the A6 class.

1. **interface** printable
2. {
3. **void** print(); }
4. **class** A6 **implements** printable{
5. **public void** print(){System.out.println("Hello");}
6. **public static void** main(String args[]){
7. A6 obj = **new** A6();
8. obj.print();
10. }
11. }

Test it Now

Output:

```
Hello
```

Java Interface Example: Drawable

In this example, the Drawable interface has only one method. Its implementation is provided by Rectangle and Circle classes. In a real scenario, an interface is defined by someone else, but its implementation is provided by different implementation providers. Moreover, it is used by someone else. The implementation part is hidden by the user who uses the interface.

File: TestInterface1.java

```
1. //Interface declaration: by first user
2. interface Drawable{
3.   void draw();
4. }
5. //Implementation: by second user
6. class Rectangle implements Drawable{
7.   public void draw(){System.out.println("drawing rectangle");}
8. }
9. class Circle implements Drawable{
10.  public void draw(){System.out.println("drawing circle");}
11. }
12. //Using interface: by third user
13. class TestInterface1{
14.  public static void main(String args[]){
15.   Drawable d=new Circle();//In real scenario, object is provided by method e.g. getDrawable()
16.   d.draw();
17. }}
```

Test it Now

Output:

```
drawing circle
```

Java Interface Example: Bank

Let's see another example of java interface which provides the implementation of Bank interface.

File: TestInterface2.java

```
1. interface Bank{
2.   float rateOfInterest();
3. }
4. class SBI implements Bank{
5.   public float rateOfInterest(){return 9.15f;}
6. }
7. class PNB implements Bank{
8.   public float rateOfInterest()
```

```

9. {return 9.7f;}9. }
10. class TestInterface2{
11. public static void main(String[] args){
12. Bank b=new SBI();
13. System.out.println("ROI: "+b.rateOfInterest());
14. }}

```

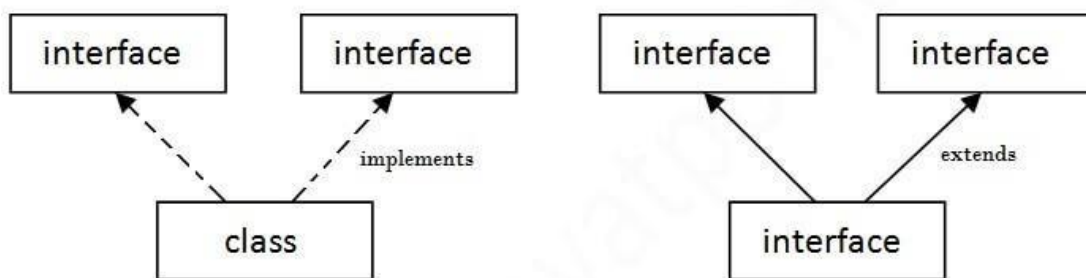
Test it Now

Output:

```
ROI: 9.15
```

Multiple inheritance in Java by interface

If a class implements multiple interfaces, or an interface extends multiple interfaces, it is known as multiple inheritance.



Multiple Inheritance in Java

```

1. interface Printable{
2. void print();
3. }
4. interface Showable{
5. void show();
6. }
7. class A7
8. implements Printable,Showable{
9. public void print(){System.out.println("Hello");}
10. public void show(){System.out.println("Welcome");}
11.
12. public static void main(String args[]){
13. A7 obj = new A7();
14. obj.print();
15. obj.show();
16. }

```

Test it Now

```
Output:Hello
        Welcome
```

Q) Multiple inheritance is not supported through class in java, but it is possible by an interface, why?

As we have explained in the inheritance chapter, multiple inheritance is not supported in the case of class because of ambiguity. However, it is supported in case of an interface because there is no ambiguity. It is because its implementation is provided by the implementation class. For example:

1. **interface** Printable{

```
2. void print();
3. }
4. interface Showable{
5. void print();
6. }
7.
8. class TestInterface3 implements Printable, Showable{
9. public void print(){System.out.println("Hello");}
10. public static void main(String args[]){
11. TestInterface3 obj = new TestInterface3();
12. obj.print();
13. }
14. }
```

Test it Now

Output:

```
Hello
```

As you can see in the above example, Printable and Showable interface have same methods but its implementation is provided by class TestInterface1, so there is no ambiguity.

Interface inheritance

A class implements an interface, but one interface extends another interface.

```
1. interface Printable{
2. void print();
3. }
4. interface Showable extends Printable{
5. void show();
6. }
7. class TestInterface4 implements Showable{
8. public void print(){System.out.println("Hello");}
9. public void show(){System.out.println("Welcome");}
10.
11. public static void main(String args[]){
```

```
12. TestInterface4 obj = new TestInterface4();
13. obj.print();
14. obj.show();
15. }
16. }
```

Test it Now

Output:

```
Hello
Welcome
```

Java 8 Default Method in Interface

Since Java 8, we can have method body in interface. But we need to make it default method. Let's see an example:

File: TestInterfaceDefault.java

```
1. interface Drawable{
2.   void draw();
3.   default void msg(){System.out.println("default method");}
4. }
5. class Rectangle implements Drawable{
6.   public void draw(){System.out.println("drawing rectangle");}
7. }
8. class TestInterfaceDefault{
9.   public static void main(String args[]){
10.    Drawable d=new Rectangle();
11.    d.draw();
12.    d.msg();
13. }}
```

Test it Now

Output:

```
drawing rectangle
default method
```

Java 8 Static Method in Interface

Since Java 8, we can have static method in interface. Let's see an example:

File: TestInterfaceStatic.java

```
1. interface Drawable{
2. void draw();
3. static int cube(int x){return x*x*x;}
4. }
5. class Rectangle implements Drawable{
6. public void draw(){System.out.println("drawing rectangle");}
7. }
8. class TestInterfaceStatic{
9. public static void main(String args[]){
10. Drawable d=new Rectangle();
11. d.draw();
12. System.out.println(Drawable.cube(3));
14. }}
```

Test it Now

Output:

```
drawing rectangle
27
```

Q) What is marker or tagged interface?

An interface which has no member is known as a marker or tagged interface, for example, Serializable, Cloneable, Remote, etc. They are used to provide some essential information to the JVM so that JVM may perform some useful operation.

```
1. //How Serializable interface is written?
2. public interface Serializable{
3. }
```

Nested Interface in Java

Note: An interface can have another interface which is known as a nested interface. We will learn it in detail in the nested classes chapter. For example:

```
1. interface printable{
2. void print();
```



```

3. interface MessagePrintable{
4.   void msg();
5. }
6. }

```

Interface and Abstract Class:

Abstract class and interface both are used to achieve abstraction where we can declare the abstract methods. Abstract class and interface both can't be instantiated. But there are many differences between abstract class and interface that are given below.

Abstract class	Interface
1) Abstract class can have abstract and non-abstract methods.	Interface can have only abstract methods. Since Java 8, it can have default and static methods also.
2) Abstract class doesn't support multiple inheritance .	Interface supports multiple inheritance .
3) Abstract class can have final, non-final, static and non-static variables .	Interface has only static and final variables .
4) Abstract class can provide the implementation of interface .	Interface can't provide the implementation of abstract class .
5) The abstract keyword is used to declare abstract class.	The interface keyword is used to declare interface.
6) An abstract class can extend another Java class and implement multiple Java interfaces.	An interface can extend another Java interface only.
7) An abstract class can be extended using keyword "extends".	An interface can be implemented using keyword "implements".

8) A Java abstract class can have class members like private, protected, etc.	Members of a Java interface are public by default.
9) Example: <pre>public abstract class Shape{ public abstract void draw(); }</pre>	Example: <pre>public interface Drawable{ void draw(); }</pre>

Simply, abstract class achieves partial abstraction (0 to 100%) whereas interface achieves fully abstraction (100%).

Example of abstract class and interface in Java

Let's see a simple example where we are using interface and abstract class both.

1. *//Creating interface that has 4 methods*
2. **interface** A{
3. **void** a();//bydefault, public and abstract
4. **void** b();
5. **void** c();
6. **void** d();
7. }
- 8.
9. *//Creating abstract class that provides the implementation of one method of A interface*
10. **abstract class** B **implements** A{
11. **public void** c(){System.out.println("I am C");}
12. }
- 13.
14. *//Creating subclass of abstract class, now we need to provide the implementation of rest of the methods*
15. **class** M **extends** B{
16. **public void** a(){System.out.println("I am a");}
17. **public void** b(){System.out.println("I am b");}
18. **public void** d(){System.out.println("I am d");}
19. }
- 20.
21. *//Creating a test class that calls the methods of A interface*
22. **class** Test5{

```
23. public static void main(String args[]){
24. A a=new M();
25. a.a();
26. a.b();
27. a.c();
28. a.d();
29. }}
```

Test it Now

Output:

```
I am a
I am b
I am c
I am d
```

Extends and Implements Together:

Inheritance is an important pillar of OOP(Object Oriented Programming). It is the mechanism in Java by which one class is allowed to inherit the features(fields and methods) of another class. There are two main keywords, “**extends**” and “**implements**” which are used in Java for inheritance. In this article, the difference between extends and implements is discussed.

Before getting into the differences, let's first understand in what scenarios each of the keywords are used.

Extends: In Java, the *extends* keyword is used to indicate that the class which is being defined is derived from the base class using inheritance. So basically, extends keyword is used to extend the functionality of the parent class to the subclass. In Java, multiple inheritances are not allowed due to ambiguity. Therefore, a class can extend only one class to avoid ambiguity.

Example:

```
class One {

    public void methodOne ()

    {
```

```
        // Some Functionality

    }

}

class Two extends One {

    public static void main(String args[])
    {

        Two t = new Two();

        // Calls the method one
        // of the above class

        t.methodOne();

    }

}
```

Implements: In Java, the *implements* keyword is used to implement an interface. An interface is a special type of class which implements a complete abstraction and only contains abstract methods. To access the interface methods, the interface must be “implemented” by another class with the implements keyword and the methods need to be implemented in the class which is inheriting the properties of the interface. Since an interface is not having the implementation of the methods, a class can implement any number of interfaces at a time.

Example

```
// Defining an interface

interface One {

    public void methodOne();
}


// Defining the second interface

interface Two {

    public void methodTwo();
}


// Implementing the two interfaces

class Three implements One, Two {

    public void methodOne()
    {

        // Implementation of the method

    }

    public void methodTwo()
    {
```

```
        // Implementation of the method

    }

}
```

Note: A class can extend a class and can implement any number of interfaces simultaneously.

Example

```
// Defining the interface

interface One {

    // Abstract method

    void methodOne();

}

// Defining a class

class Two {

    // Defining a method

    public void methodTwo()

    {

    }

}
```

```

}

// Class which extends the class Two
// and implements the interface One

class Three extends Two implements One {

    public void methodOne()

    {

        // Implementation of the method

    }

}

```

Note: An interface can extend any number of interfaces at a time.
Example:

```

// Defining the interface One

interface One {

    void methodOne();

}

// Defining the interface Two

interface Two {

```

```

        void methodTwo();
    }

    // Interface extending both the
    // defined interfaces
    interface Three extends One, Two {
    }

```

The following table explains the difference between the extends and interface:

S.No.	Extends	Implements
1.	By using “extends” keyword a class can inherit another class, or an interface can inherit other interfaces	By using “implements” keyword a class can implement an interface
2.	It is not compulsory that subclass that extends a superclass override all the methods in a superclass.	It is compulsory that class implementing an interface has to implement all the methods of that interface.
3.	Only one superclass can be extended by a class.	A class can implement any number of an interface at a time
4.	Any number of interfaces can be extended by interface.	An interface can never implement any other interface

EXCEPTION HANDLING

Exception:

The **Exception Handling in Java** is one of the powerful *mechanism to handle the runtime errors* so that the normal flow of the application can be maintained.

In this tutorial, we will learn about Java exceptions, its types, and the difference between checked and unchecked exceptions.

What is Exception in Java?

Dictionary Meaning: Exception is an abnormal condition.

In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

What is Exception Handling?

Exception Handling is a mechanism to handle runtime errors such as `ClassNotFoundException`, `IOException`, `SQLException`, `RemoteException`, etc.

Advantage of Exception Handling

The core advantage of exception handling is **to maintain the normal flow of the application**. An exception normally disrupts the normal flow of the application; that is why we need to handle exceptions. Let's consider a scenario:

1. statement 1;
2. statement 2;
3. statement 3;
4. statement 4;
5. statement 5; *//exception occurs*
6. statement 6;
7. statement 7;
8. statement 8;
9. statement 9;
10. statement 10;

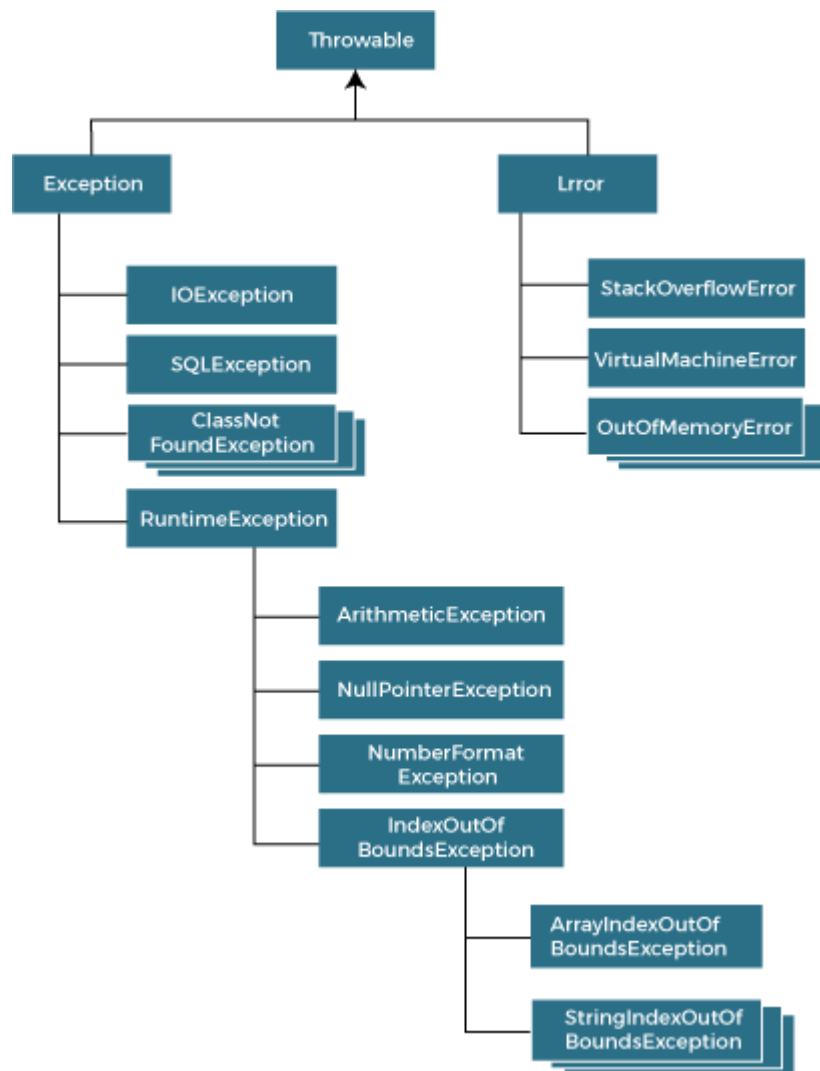
Suppose there are 10 statements in a Java program and an exception occurs at statement 5; the rest of the code will not be executed, i.e., statements 6 to 10 will not be executed. However, when we perform exception handling, the rest of the statements will be executed. That is why we use exception handling in [Java](#).

Do You Know?

- What is the difference between checked and unchecked exceptions?
- What happens behind the code `int data=50/0;`?
- Why use multiple catch block?
- Is there any possibility when the finally block is not executed?
- What is exception propagation?
- What is the difference between the throw and throws keyword?
- What are the 4 rules for using exception handling with method overriding?

Hierarchy of Java Exception classes

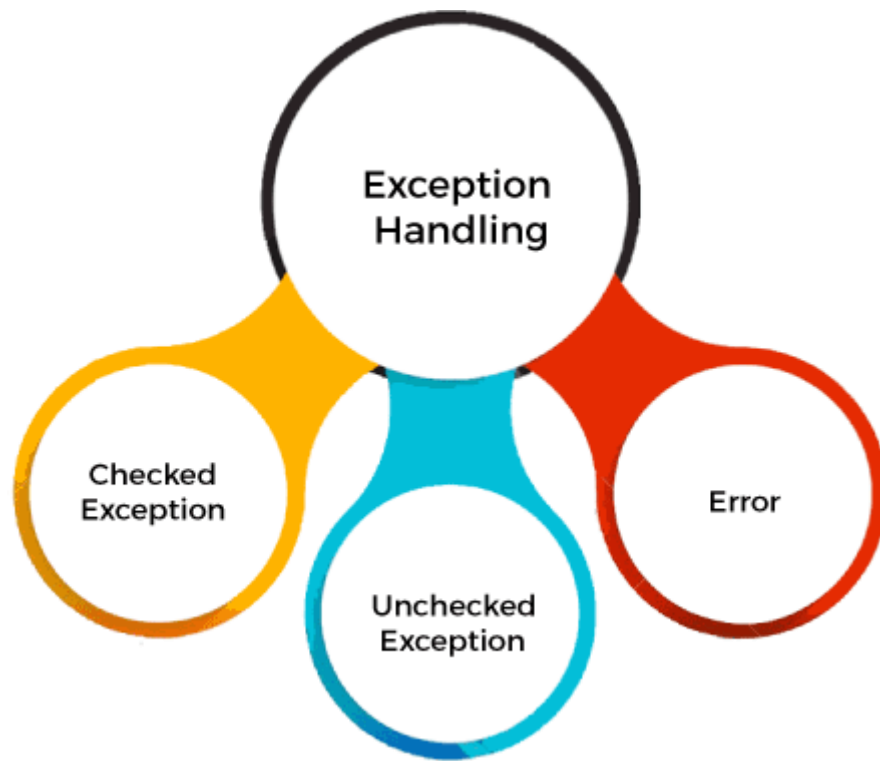
The `java.lang.Throwable` class is the root class of Java Exception hierarchy inherited by two subclasses: `Exception` and `Error`. The hierarchy of Java Exception classes is given below:



Types of Java Exceptions

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:

1. Checked Exception
2. Unchecked Exception
3. Error



Difference between Checked and Unchecked Exceptions

1) Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error

Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

Java Exception Keywords

Java provides five keywords that are used to handle the exception. The following table describes each.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It must be followed by either catch or finally block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which handles the exception. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether the exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception. It is always used with method signature.

Java Exception Handling Example

Let's see an example of Java Exception Handling in which we are using a try-catch statement to handle the exception.

JavaExceptionExample.java

```

1. public class JavaExceptionExample{
2.     public static void main(String args[]){
3.         try{
4.             //code that may raise exception
5.             int data=100/0;
6.         }catch(ArithmeticException e){System.out.println(e);}
7.         //rest code of the program
8.         System.out.println("rest of the code...");
9.     }
10. }
```

Test it Now

Output:

```
Exception in thread main java.lang.ArithmeticException: / by zero
rest of the code...
```

In the above example, 100/0 raises an `ArithmeticException` which is handled by a try-catch block.

Common Scenarios of Java Exceptions

There are given some scenarios where unchecked exceptions may occur. They are as follows:

1) A scenario where `ArithmeticException` occurs

If we divide any number by zero, there occurs an `ArithmeticException`.

1. `int a=50/0;//ArithmeticException`

2) A scenario where `NullPointerException` occurs

If we have a null value in any variable, performing any operation on the variable throws a `NullPointerException`.

1. `String s=null;`
2. `System.out.println(s.length());//NullPointerException`

3) A scenario where `NumberFormatException` occurs

If the formatting of any variable or number is mismatched, it may result into `NumberFormatException`. Suppose we have a string variable that has characters; converting this variable into digit will cause `NumberFormatException`.

1. `String s="abc";`
2. `int i=Integer.parseInt(s);//NumberFormatException`

4) A scenario where `ArrayIndexOutOfBoundsException` occurs

When an array exceeds to its size, the `ArrayIndexOutOfBoundsException` occurs. There may be other reasons to occur `ArrayIndexOutOfBoundsException`. Consider the following statements.

1. `int a[]=new int[5];`
2. `a[10]=50; //ArrayIndexOutOfBoundsException`

Using Try Catch:

Java try block

Java **try** block is used to enclose the code that might throw an exception. It must be used within the method.

If an exception occurs at the particular statement in the try block, the rest of the block code will not execute. So, it is recommended not to keep the code in try block that will not throw an exception.

Java try block must be followed by either catch or finally block.

Syntax of Java try-catch

1. **try**{
2. //code that may throw an exception
3. }**catch**(Exception_class_Name ref){}

Syntax of try-finally block

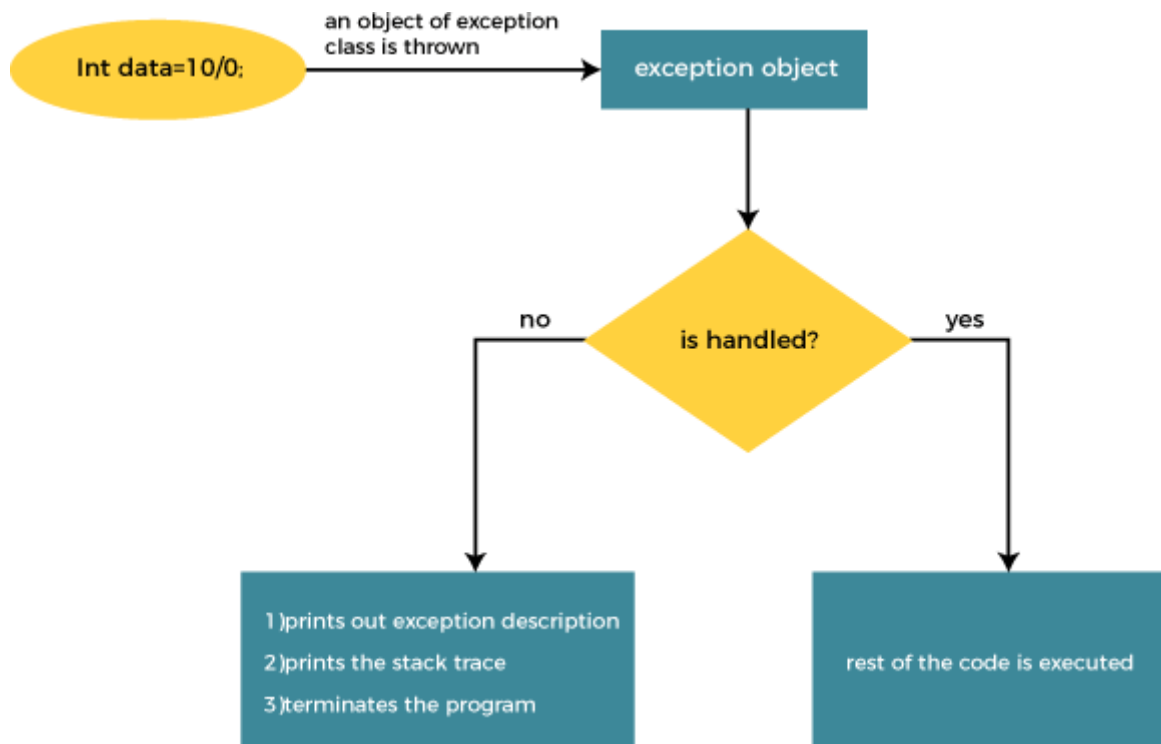
1. **try**{
2. //code that may throw an exception
3. }**finally**{}

Java catch block

Java catch block is used to handle the Exception by declaring the type of exception within the parameter. The declared exception must be the parent class exception (i.e., Exception) or the generated exception type. However, the good approach is to declare the generated type of exception.

The catch block must be used after the try block only. You can use multiple catch block with a single try block.

Internal Working of Java try-catch block



The JVM firstly checks whether the exception is handled or not. If exception is not handled, JVM provides a default exception handler that performs the following tasks:

- Prints out exception description.
- Prints the stack trace (Hierarchy of methods where the exception occurred).
- Causes the program to terminate.

But if the application programmer handles the exception, the normal flow of the application is maintained, i.e., rest of the code is executed.

Problem without exception handling

Let's try to understand the problem if we don't use a try-catch block.

Example 1

TryCatchExample1.java

1. **public class** TryCatchExample1 {
- 2.
3. **public static void** main(String[] args) {
- 4.
5. **int** data=50/0; *//may throw exception*


```
6.  
7.     System.out.println("rest of the code");  
8.  
9. }  
10.  
11. }
```

Test it Now

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

As displayed in the above example, the **rest of the code** is not executed (in such case, the **rest of the code** statement is not printed).

There might be 100 lines of code after the exception. If the exception is not handled, all the code below the exception won't be executed.

Solution by exception handling

Let's see the solution of the above problem by a java try-catch block.

Example 2

TryCatchExample2.java

```
1. public class TryCatchExample2 {  
2.  
3.     public static void main(String[] args) {  
4.         try  
5.         {  
6.             int data=50/0; //may throw exception  
7.         }  
8.         //handling the exception  
9.         catch(ArithmeticException e)  
10.        {  
11.            System.out.println(e);  
12.        }  
13.        System.out.println("rest of the code");  
14.    }  
15. }
```

16. }

Test it Now

Output:

```
java.lang.ArithmeticException: / by zero
rest of the code
```

As displayed in the above example, the **rest of the code** is executed, i.e., the **rest of the code** statement is printed.

Example 3

In this example, we also kept the code in a try block that will not throw an exception.

TryCatchExample3.java

```
1. public class TryCatchExample3 {
2.
3.     public static void main(String[] args) {
4.         try
5.         {
6.             int data=50/0; //may throw exception
7.             // if exception occurs, the remaining statement will not execute
8.             System.out.println("rest of the code");
9.         }
10.        // handling the exception
11.        catch(ArithmeticException e)
12.        {
13.            System.out.println(e);
14.        }
15.
16.    }
17.
18. }
```

Test it Now

Output:

```
java.lang.ArithmeticException: / by zero
```

Here, we can see that if an exception occurs in the try block, the rest of the block code will not execute.

Example 4

Here, we handle the exception using the parent class exception.

TryCatchExample4.java

```
1. public class TryCatchExample4 {
2.
3.     public static void main(String[] args) {
4.         try
5.         {
6.             int data=50/0; //may throw exception
7.         }
8.         // handling the exception by using Exception class
9.         catch(Exception e)
10.        {
11.            System.out.println(e);
12.        }
13.        System.out.println("rest of the code");
14.    }
15.
16. }
```

Test it Now

Output:

```
java.lang.ArithmeticException: / by zero
rest of the code
```

Example 5

Let's see an example to print a custom message on exception.

TryCatchExample5.java

```
1. public class TryCatchExample5 {
2.
3.     public static void main(String[] args) {
4.         try
5.         {
6.             int data=50/0; //may throw exception
7.         }
```

```
8.         // handling the exception
9.     catch(Exception e)
10.    {
11.        // displaying the custom message
12.        System.out.println("Can't divided by zero");
13.    }
14. }
15.
16. }
```

Test it Now

Output:

```
Can't divided by zero
```

Example 6

Let's see an example to resolve the exception in a catch block.

TryCatchExample6.java

```
1. public class TryCatchExample6 {
2.
3.     public static void main(String[] args) {
4.         int i=50;
5.         int j=0;
6.         int data;
7.         try
8.         {
9.             data=i/j; //may throw exception
10.        }
11.        // handling the exception
12.        catch(Exception e)
13.        {
14.            // resolving the exception in catch block
15.            System.out.println(i/(j+2));
16.        }
17.    }
18. }
```

Catching Multiple Exception:

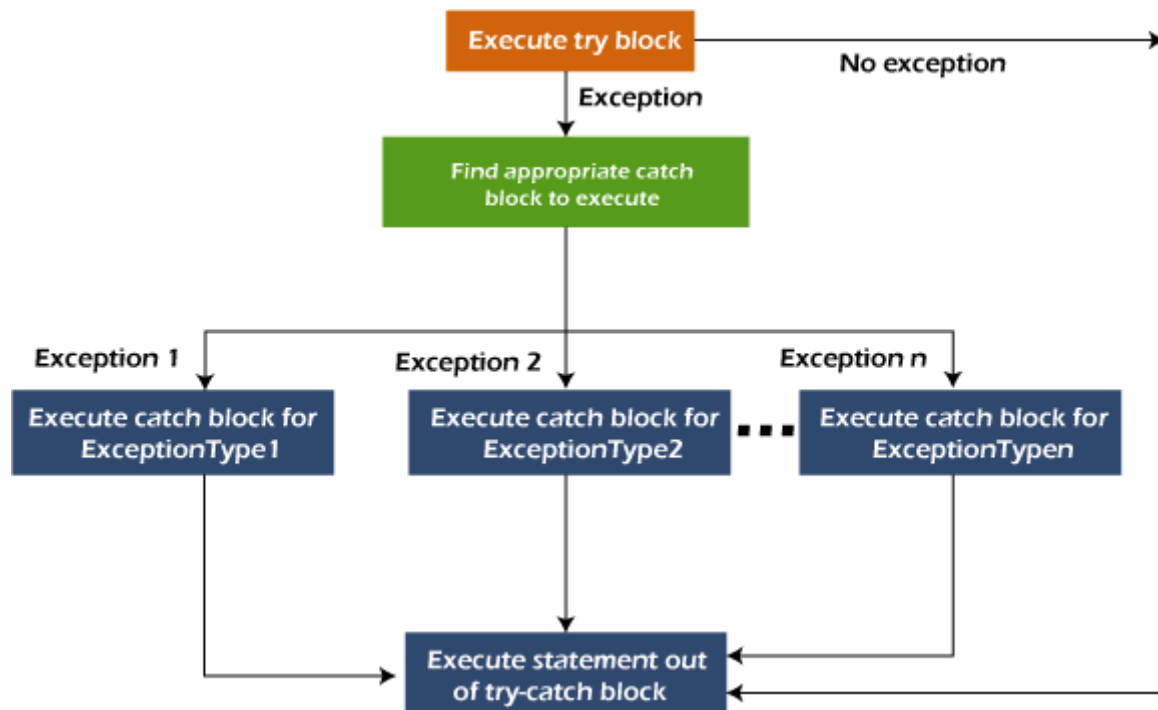
Java Multi-catch block

A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.

Points to remember

- At a time only one exception occurs and at a time only one catch block is executed.
- All catch blocks must be ordered from most specific to most general, i.e. catch for `ArithmeticException` must come before catch for `Exception`.

Flowchart of Multi-catch Block



Example 1

Let's see a simple example of java multi-catch block.

MultipleCatchBlock1.java

```
1. public class MultipleCatchBlock1 {
```

```

2.
3.   public static void main(String[] args) {
4.
5.       try{
6.           int a[]=new int[5];
7.           a[5]=30/0;
8.       }
9.       catch(ArithmeticException e)
10.          {
11.              System.out.println("Arithmetic Exception occurs");
12.          }
13.       catch(ArrayIndexOutOfBoundsException e)
14.          {
15.              System.out.println("ArrayIndexOutOfBoundsException occurs");
16.          }
17.       catch(Exception e)
18.          {
19.              System.out.println("Parent Exception occurs");
20.          }
21.       System.out.println("rest of the code");
22.   }
23. }

```

Test it Now

Output:

```

Arithmetic Exception occurs
rest of the code

```

Example 2

MultipleCatchBlock2.java

```

1. public class MultipleCatchBlock2 {
2.
3.   public static void main(String[] args) {
4.
5.       try{
6.           int a[]=new int[5];
7.

```

```

8.         System.out.println(a[10]);
9.     }
10.    catch(ArithmeticException e)
11.    {
12.        System.out.println("Arithmetic Exception occurs");
13.    }
14.    catch(ArrayIndexOutOfBoundsException e)
15.    {
16.        System.out.println("ArrayIndexOutOfBoundsException occurs");
17.    }
18.    catch(Exception e)
19.    {
20.        System.out.println("Parent Exception occurs");
21.    }
22.    System.out.println("rest of the code");
23. }
24. }

```

Test it Now

Output:

```

ArrayIndexOutOfBoundsException occurs
rest of the code

```

In this example, try block contains two exceptions. But at a time only one exception occurs and its corresponding catch block is executed.

MultipleCatchBlock3.java

```

1. public class MultipleCatchBlock3 {
2.
3.    public static void main(String[] args) {
4.
5.        try{
6.            int a[]=new int[5];
7.            a[5]=30/0;
8.            System.out.println(a[10]);
9.        }
10.       catch(ArithmeticException e)
11.       {

```

```

12.         System.out.println("Arithmetic Exception occurs");
13.     }
14.     catch(ArrayIndexOutOfBoundsException e)
15.     {
16.         System.out.println("ArrayIndexOutOfBoundsException occurs");
17.     }
18.     catch(Exception e)
19.     {
20.         System.out.println("Parent Exception occurs");
21.     }
22.     System.out.println("rest of the code");
23. }
24. }

```

Test it Now

Output:

```

Arithmetic Exception occurs
rest of the code

```

Example 4

In this example, we generate `NullPointerException`, but didn't provide the corresponding exception type. In such case, the catch block containing the parent exception class **Exception** will be invoked.

MultipleCatchBlock4.java

```

1. public class MultipleCatchBlock4 {
2.
3.     public static void main(String[] args) {
4.
5.         try{
6.             String s=null;
7.             System.out.println(s.length());
8.         }
9.         catch(ArithmeticException e)
10.        {
11.            System.out.println("Arithmetic Exception occurs");
12.        }
13.        catch(ArrayIndexOutOfBoundsException e)

```



```

14.     {
15.         System.out.println("ArrayIndexOutOfBoundsException occurs");
16.     }
17.     catch(Exception e)
18.     {
19.         System.out.println("Parent Exception occurs");
20.     }
21.     System.out.println("rest of the code");
22. }
23. }

```

Test it Now

Output:

```

Parent Exception occurs
rest of the code

```

Example 5

Let's see an example, to handle the exception without maintaining the order of exceptions (i.e. from most specific to most general).

MultipleCatchBlock5.java

```

1. class MultipleCatchBlock5{
2.     public static void main(String args[]){
3.         try{
4.             int a[]=new int[5];
5.             a[5]=30/0;
6.         }
7.         catch(Exception e){System.out.println("common task completed");}
8.         catch(ArithmeticException e){System.out.println("task1 is completed");}
9.         catch(ArrayIndexOutOfBoundsException e){System.out.println("task 2 completed");}
10.        System.out.println("rest of the code...");
11.    }
12. }

```

Test it Now

Output:

```

Compile-time error

```

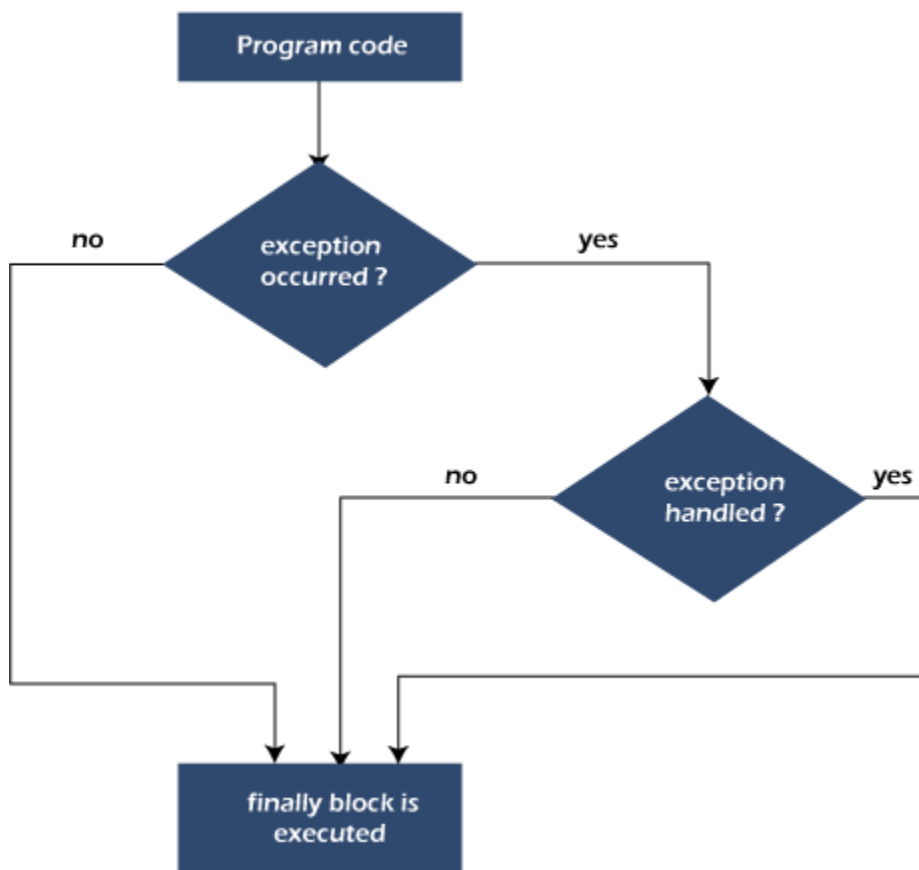
Using Finally Clause:

Java finally block is a block used to execute important code such as closing the connection, etc.

Java finally block is always executed whether an exception is handled or not. Therefore, it contains all the necessary statements that need to be printed regardless of the exception occurs or not.

The finally block follows the try-catch block.

Flowchart of finally block



Note: If you don't handle the exception, before terminating the program, JVM executes finally block (if any).

Why use Java finally block?

- finally block in Java can be used to put "**cleanup**" code such as closing a file, closing connection, etc.
- The important statements to be printed can be placed in the finally block.

Usage of Java finally

Let's see the different cases where Java finally block can be used.

Case 1: When an exception does not occur

Let's see the below example where the Java program does not throw any exception, and the finally block is executed after the try block.

TestFinallyBlock.java

```
1. class TestFinallyBlock {
2.     public static void main(String args[]){
3.         try{
4.             //below code do not throw any exception
5.             int data=25/5;
6.             System.out.println(data);
7.         }
8.         //catch won't be executed
9.         catch(NullPointerException e){
10.            System.out.println(e);
11.        }
12.        //executed regardless of exception occurred or not
13.        finally {
14.            System.out.println("finally block is always executed");
15.        }
16.
17.        System.out.println("rest of the code...");
18.    }
19. }
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestFinallyBlock.java
C:\Users\Anurati\Desktop\abcDemo>java TestFinallyBlock
5
finally block is always executed
rest of the code...
```

Case 2: When an exception occurs but not handled by the catch block

Let's see the following example. Here, the code throws an exception however the catch block cannot handle it. Despite this, the finally block is executed after the try block and then the program terminates abnormally.

TestFinallyBlock1.java

```
1. public class TestFinallyBlock1{
2.     public static void main(String args[]){
3.
4.         try {
5.
6.             System.out.println("Inside the try block");
7.
8.             //below code throws divide by zero exception
9.             int data=25/0;
10.            System.out.println(data);
11.        }
12.        //cannot handle Arithmetic type exception
13.        //can only accept Null Pointer type exception
14.        catch(NullPointerException e){
15.            System.out.println(e);
16.        }
17.
18.        //executes regardless of exception occurred or not
19.        finally {
20.            System.out.println("finally block is always executed");
21.        }
22.
23.        System.out.println("rest of the code...");
24.    }
25. }
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestFinallyBlock1.java

C:\Users\Anurati\Desktop\abcDemo>java TestFinallyBlock1
Inside the try block
finally block is always executed
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at TestFinallyBlock1.main(TestFinallyBlock1.java:9)
```

Case 3: When an exception occurs and is handled by the catch block

Example:

Let's see the following example where the Java code throws an exception and the catch block handles the exception. Later the finally block is executed after the try-catch block. Further, the rest of the code is also executed normally.

TestFinallyBlock2.java

```
1. public class TestFinallyBlock2{
2.     public static void main(String args[]){
3.
4.         try {
5.
6.             System.out.println("Inside try block");
7.
8.             //below code throws divide by zero exception
9.             int data=25/0;
10.            System.out.println(data);
11.        }
12.
13.        //handles the Arithmetic Exception / Divide by zero exception
14.        catch(ArithmeticException e){
15.            System.out.println("Exception handled");
16.            System.out.println(e);
17.        }
18.
19.        //executes regardless of exception occurred or not
20.        finally {
21.            System.out.println("finally block is always executed");
22.        }
```

```
23.  
24.    System.out.println("rest of the code...");  
25.    }  
26.    }
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestFinallyBlock2.java  
  
C:\Users\Anurati\Desktop\abcDemo>java TestFinallyBlock2  
Inside try block  
Exception handled  
java.lang.ArithmeticException: / by zero  
finally block is always executed  
rest of the code...
```

Rule: For each try block there can be zero or more catch blocks, but only one finally block.

Note: The finally block will not be executed if the program exits (either by calling `System.exit()` or by causing a fatal error that causes the process to abort).

Throwing Exception:

In Java, exceptions allows us to write good quality codes where the errors are checked at the compile time instead of runtime and we can create custom exceptions making the code recovery and debugging easier.

Java throw keyword

The Java throw keyword is used to throw an exception explicitly.

We specify the **exception** object which is to be thrown. The Exception has some message with it that provides the error description. These exceptions may be related to user inputs, server, etc.

We can throw either checked or unchecked exceptions in Java by throw keyword. It is mainly used to throw a custom exception. We will discuss custom exceptions later in this section.

We can also define our own set of conditions and throw an exception explicitly using throw keyword. For example, we can throw `ArithmeticException` if we divide a number by another number. Here, we just need to set the condition and throw exception using throw keyword.

The syntax of the Java throw keyword is given below.

throw Instance i.e.,

1. **throw new** exception_class("error message");

Let's see the example of throw IOException.

1. **throw new** IOException("sorry device error");

Where the Instance must be of type Throwable or subclass of Throwable. For example, Exception is the sub class of Throwable and the user-defined exceptions usually extend the Exception class.

Java throw keyword Example

Example 1: Throwing Unchecked Exception

In this example, we have created a method named validate() that accepts an integer as a parameter. If the age is less than 18, we are throwing the ArithmeticException otherwise print a message welcome to vote.

TestThrow1.java

In this example, we have created the validate method that takes integer value as a parameter. If the age is less than 18, we are throwing the ArithmeticException otherwise print a message welcome to vote.

1. **public class** TestThrow1 {
2. //function to check if person is eligible to vote or not
3. **public static void** validate(**int** age) {
4. **if**(age<18) {
5. //throw Arithmetic exception if not eligible to vote
6. **throw new** ArithmeticException("Person is not eligible to vote");
7. }
8. **else** {
9. System.out.println("Person is eligible to vote!!");
10. }
11. }
12. //main method
13. **public static void** main(String args[]){
14. //calling the function
15. validate(13);

```
16.     System.out.println("rest of the code...");
17. }
18. }
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestThrow1.java

C:\Users\Anurati\Desktop\abcDemo>java TestThrow1
Exception in thread "main" java.lang.ArithmeticException: Person is not eligible to
vote
    at TestThrow1.validate(TestThrow1.java:8)
    at TestThrow1.main(TestThrow1.java:18)
```

The above code throw an unchecked exception. Similarly, we can also throw unchecked and user defined exceptions.

Note: If we throw unchecked exception from a method, it is must to handle the exception or declare in throws clause.

If we throw a checked exception using throw keyword, it is must to handle the exception using catch block or the method must declare it using throws declaration.

Example 2: Throwing Checked Exception

Note: Every subclass of Error and RuntimeException is an unchecked exception in Java. A checked exception is everything else under the Throwable class.

TestThrow2.java

```
1. import java.io.*;
2.
3. public class TestThrow2 {
4.
5.     //function to check if person is eligible to vote or not
6.     public static void method() throws FileNotFoundException {
7.
8.         FileReader file = new FileReader("C:\\Users\\Anurati\\Desktop\\abc.txt");
9.         BufferedReader fileInput = new BufferedReader(file);
10.
11.
12.         throw new FileNotFoundException();
13.     }
```



```

14. }
15. //main method
16. public static void main(String args[]){
17.     try
18.     {
19.         method();
20.     }
21.     catch (FileNotFoundException e)
22.     {
23.         e.printStackTrace();
24.     }
25.     System.out.println("rest of the code...");
26. }
27. }

```

Output:

```

C:\Users\Anurati\Desktop\abcDemo>javac TestThrow2.java

C:\Users\Anurati\Desktop\abcDemo>java TestThrow2
java.io.FileNotFoundException
    at TestThrow2.method(TestThrow2.java:12)
    at TestThrow2.main(TestThrow2.java:22)
rest of the code...

```

Example 3: Throwing User-defined Exception

exception is everything else under the Throwable class.

TestThrow3.java

```

1. // class represents user-defined exception
2. class UserDefinedException extends Exception
3. {
4.     public UserDefinedException(String str)
5.     {
6.         // Calling constructor of parent Exception
7.         super(str);
8.     }
9. }
10. // Class that uses above MyException
11. public class TestThrow3

```

```

12. {
13.     public static void main(String args[])
14.     {
15.         try
16.         {
17.             // throw an object of user defined exception
18.             throw new UserDefinedException("This is user-defined exception");
19.         }
20.         catch (UserDefinedException ude)
21.         {
22.             System.out.println("Caught the exception");
23.             // Print the message from MyException object
24.             System.out.println(ude.getMessage());
25.         }
26.     }
27. }

```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestThrow3.java
```

```
C:\Users\Anurati\Desktop\abcDemo>java TestThrow3
```

```
Caught the exception:
```

```
This is user-defined exception
```

In Java, we can create our own exceptions that are derived classes of the Exception class. Creating our own Exception is known as custom exception or user-defined exception. Basically, Java custom exceptions are used to customize the exception according to user need.

Consider the example 1 in which InvalidAgeException class extends the Exception class.

Using the custom exception, we can have your own exception and message. Here, we have passed a string to the constructor of superclass i.e. Exception class that can be obtained using getMessage() method on the object we have created.

In this section, we will learn how custom exceptions are implemented and used in Java programs.

Why use custom exceptions?

Java exceptions cover almost all the general type of exceptions that may occur in the programming. However, we sometimes need to create custom exceptions.

Following are few of the reasons to use custom exceptions:

- To catch and provide specific treatment to a subset of existing Java exceptions.
- Business logic exceptions: These are the exceptions related to business logic and workflow. It is useful for the application users or the developers to understand the exact problem.

In order to create custom exception, we need to extend Exception class that belongs to java.lang package.

Consider the following example, where we create a custom exception named WrongFileNameException:

```
1. public class WrongFileNameException extends Exception {  
2.     public WrongFileNameException(String errorMessage) {  
3.         super(errorMessage);  
4.     }  
5. }
```

Note: We need to write the constructor that takes the String as the error message and it is called parent class constructor.

Example 1:

Let's see a simple example of Java custom exception. In the following code, constructor of InvalidAgeException takes a string as an argument. This string is passed to constructor of parent class Exception using the super() method. Also the constructor of Exception class can be called without using a parameter and calling super() method is not mandatory.

TestCustomException1.java

```
1. // class representing custom exception  
2. class InvalidAgeException extends Exception  
3. {  
4.     public InvalidAgeException (String str)  
5.     {  
6.         // calling the constructor of parent Exception
```

```
7.     super(str);8.
    }
9. }
10.
11. // class that uses custom exception InvalidAgeException
12. public class TestCustomException1
13. {
14.
15.     // method to check the age
16.     static void validate (int age) throws InvalidAgeException{
17.         if(age < 18){
18.
19.             // throw an object of user defined exception
20.             throw new InvalidAgeException("age is not valid to vote");
21.         }
22.         else {
23.             System.out.println("welcome to vote");
24.         }
25.     }
26.
27.     // main method
28.     public static void main(String args[])
29.     {
30.         try
31.         {
32.             // calling the method
33.             validate(13);
34.         }
35.         catch (InvalidAgeException ex)
36.         {
37.             System.out.println("Caught the exception");
38.
39.             // printing the message from InvalidAgeException object
40.             System.out.println("Exception occurred: " + ex);
41.         }
42.
43.         System.out.println("rest of the code...");
```

```
44. }  
45. }
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestCustomException1.java  
  
C:\Users\Anurati\Desktop\abcDemo>java TestCustomException1  
Caught the exception  
Exception occurred: InvalidAgeException: age is not valid to vote  
rest of the code...
```

Example 2:

TestCustomException2.java

```
1. // class representing custom exception  
2. class MyCustomException extends Exception  
3. {  
4.  
5. }  
6.  
7. // class that uses custom exception MyCustomException  
8. public class TestCustomException2  
9. {  
10. // main method  
11. public static void main(String args[])  
12. {  
13.     try  
14.     {  
15.         // throw an object of user defined exception  
16.         throw new MyCustomException();  
17.     }  
18.     catch (MyCustomException ex)  
19.     {  
20.         System.out.println("Caught the exception");  
21.         System.out.println(ex.getMessage());  
22.     }  
23.  
24.     System.out.println("rest of the code...");
```

25. }

26. }

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestCustomException2.java
```

```
C:\Users\Anurati\Desktop\abcDemo>java TestCustomException2
```

```
Caught the exception
```

```
null
```

```
rest of the code...
```

UNIT - IV

MULTITHREADING

Introduction:

Multithreading is a Java feature that allows concurrent execution of two or more parts of a program for maximum utilization of CPU. Each part of such program is called a thread. So, threads are light-weight processes within a process.

Threads can be created by using two mechanisms :

1. Extending the Thread class
2. Implementing the Runnable Interface

Thread creation by extending the Thread class We create a class that extends the **java.lang.Thread** class. This class overrides the **run()** method available in the Thread class. A thread begins its life inside **run()** method. We create an object of our new class and call **start()** method to start the execution of a thread. **Start()** invokes the **run()** method on the Thread object.

- Java

```
// Java code for thread creation by extending

// the Thread class

class MultithreadingDemo extends Thread {
    public void run()
    {

        try {

            // Displaying the thread that is running
            System.out.println("Thread " +
                Thread.currentThread().getId()
                + " is running");

        }

        catch (Exception e) {

            // Throwing an exception
            System.out.println("Exception is caught");
        }

    }

}
```



```
// Main Class

public class Multithread {

    public static void main(String[] args)

    {

        int n = 8; // Number of threads

        for (int i = 0; i < n; i++) {

            MultithreadingDemo object

                = new MultithreadingDemo();

            object.start();

        }

    }

}
```

Output

Thread 15 is running

Thread 14 is running

Thread 16 is running

Thread 12 is running

Thread 11 is running

Thread 13 is running

Thread 18 is running

Thread 17 is running

Thread creation by implementing the Runnable Interface

We create a new class which implements java.lang.Runnable interface and override run() method. Then we instantiate a Thread object and call start() method on this object.

- **Java**

```
// Java code for thread creation by implementing
// the Runnable Interface

class MultithreadingDemo implements Runnable {

    public void run()

    {

        try {

            // Displaying the thread that is running

            System.out.println(

                "Thread " + Thread.currentThread().getId()

                + " is running");

        }

        catch (Exception e) {

            // Throwing an exception
```

```
        System.out.println("Exception is caught");
    }

}

// Main Class

class Multithread {

    public static void main(String[] args)
    {

        int n = 8; // Number of threads

        for (int i = 0; i < n; i++) {

            Thread object
                = new Thread(new MultithreadingDemo());

            object.start();

        }

    }

}
```

Output

```
Thread 13 is running
Thread 11 is running
Thread 12 is running
Thread 15 is running
Thread 14 is running
Thread 18 is running
Thread 17 is running
```

Thread 16 is running

Thread Class vs Runnable Interface

1. If we extend the Thread class, our class cannot extend any other class because Java doesn't support multiple inheritance. But, if we implement the Runnable interface, our class can still extend other base classes.
2. We can achieve basic functionality of a thread by extending Thread class because it provides some inbuilt methods like `yield()`, `interrupt()` etc. that are not available in Runnable interface.
3. Using runnable will give you an object that can be shared amongst multiple threads.

Multithreading in [Java](#) is a process of executing multiple threads simultaneously.

A thread is a lightweight sub-process, the smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.

However, we use multithreading than multiprocessing because threads use a shared memory area. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.

Java Multithreading is mostly used in games, animation, etc.

Advantages of Java Multithreading

- 1) It **doesn't block the user** because threads are independent and you can perform multiple operations at the same time.
- 2) You **can perform many operations together, so it saves time**.
- 3) Threads are **independent**, so it doesn't affect other threads if an exception occurs in a single thread.

Multitasking

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved in two ways:

- Process-based Multitasking (Multiprocessing)
- Thread-based Multitasking (Multithreading)

1) Process-based Multitasking (Multiprocessing)

- Each process has an address in memory. In other words, each process allocates a separate memory area.
- A process is heavyweight.
- Cost of communication between the process is high.
- Switching from one process to another requires some time for saving and loading registers, memory maps, updating lists, etc.

2) Thread-based Multitasking (Multithreading)

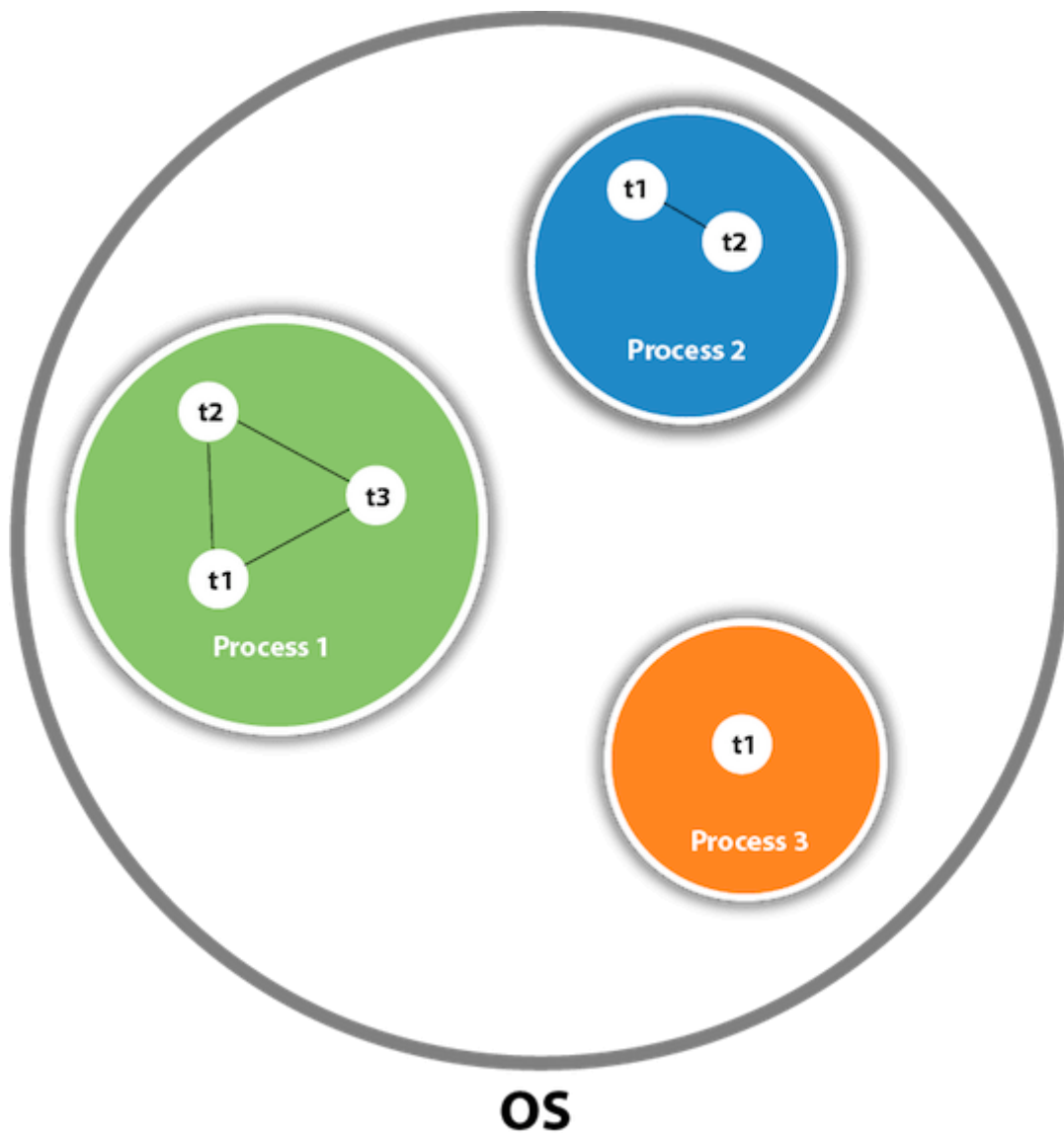
- Threads share the same address space.
- A thread is lightweight.
- Cost of communication between the thread is low.

Note: At least one process is required for each thread.

What is Thread in java

A thread is a lightweight subprocess, the smallest unit of processing. It is a separate path of execution.

Threads are independent. If there occurs exception in one thread, it doesn't affect other threads. It uses a shared memory area.



As shown in the above figure, a thread is executed inside the process. There is context-switching between the threads. There can be multiple processes inside the OS, and one process can have multiple threads.

Note: At a time one thread is executed only.

Java Thread class

Java provides **Thread class** to achieve thread programming. Thread class provides constructors and methods to create and perform operations on a thread. Thread class extends Object class and implements Runnable interface.

Java Thread Methods

S.N .	Modifier and Type	Method	Description
1)	void	<u>start()</u>	It is used to start the execution of the thread.
2)	void	<u>run()</u>	It is used to do an action for a thread.
3)	static void	<u>sleep()</u>	It sleeps a thread for the specified amount of time.
4)	static Thread	<u>currentThread()</u>	It returns a reference to the currently executing thread object.
5)	void	<u>join()</u>	It waits for a thread to die.
6)	int	<u>getPriority()</u>	It returns the priority of the thread.
7)	void	<u>setPriority()</u>	It changes the priority of the thread.
8)	String	<u>getName()</u>	It returns the name of the thread.

9)	void	<u>setName()</u>	It changes the name of the thread.
10)	long	<u>getId()</u>	It returns the id of the thread.
11)	boolean	<u>isAlive()</u>	It tests if the thread is alive.
12)	static void	<u>yield()</u>	It causes the currently executing thread object to pause and allow other threads to execute temporarily.
13)	void	<u>suspend()</u>	It is used to suspend the thread.
14)	void	<u>resume()</u>	It is used to resume the suspended thread.
15)	void	<u>stop()</u>	It is used to stop the thread.
16)	void	<u>destroy()</u>	It is used to destroy the thread group and all of its subgroups.

17)	boolean	<u>isDaemon()</u>	It tests if the thread is a daemon thread.
18)	void	<u>setDaemon()</u>	It marks the thread as daemon or user thread.
19)	void	<u>interrupt()</u>	It interrupts the thread.
20)	boolean	<u>isInterrupted()</u>	It tests whether the thread has been interrupted.
21)	static boolean	<u>interrupted()</u>	It tests whether the current thread has been interrupted.
22)	static int	<u>activeCount()</u>	It returns the number of active threads in the current thread's thread group.
23)	void	<u>checkAccess()</u>	It determines if the currently running thread has permission

			to modify the thread.
24)	static boolean	<u>holdLock()</u>	It returns true if and only if the current thread holds the monitor lock on the specified object.
25)	static void	<u>dumpStack()</u>	It is used to print a stack trace of the current thread to the standard error stream.
26)	StackTraceElement[]	<u>getStackTrace()</u>	It returns an array of stack trace elements representing the stack dump of the thread.
27)	static int	<u>enumerate()</u>	It is used to copy every active thread's thread group and its subgroup into the specified array.

28)	Thread.State	<u>getState()</u>	It is used to return the state of the thread.
29)	ThreadGroup	<u>getThreadGroup()</u>	It is used to return the thread group to which this thread belongs
30)	String	<u>toString()</u>	It is used to return a string representation of this thread, including the thread's name, priority, and thread group.
31)	void	<u>notify()</u>	It is used to give the notification for only one thread which is waiting for a particular object.
32)	void	<u>notifyAll()</u>	It is used to give the notification to all waiting threads of a particular object.

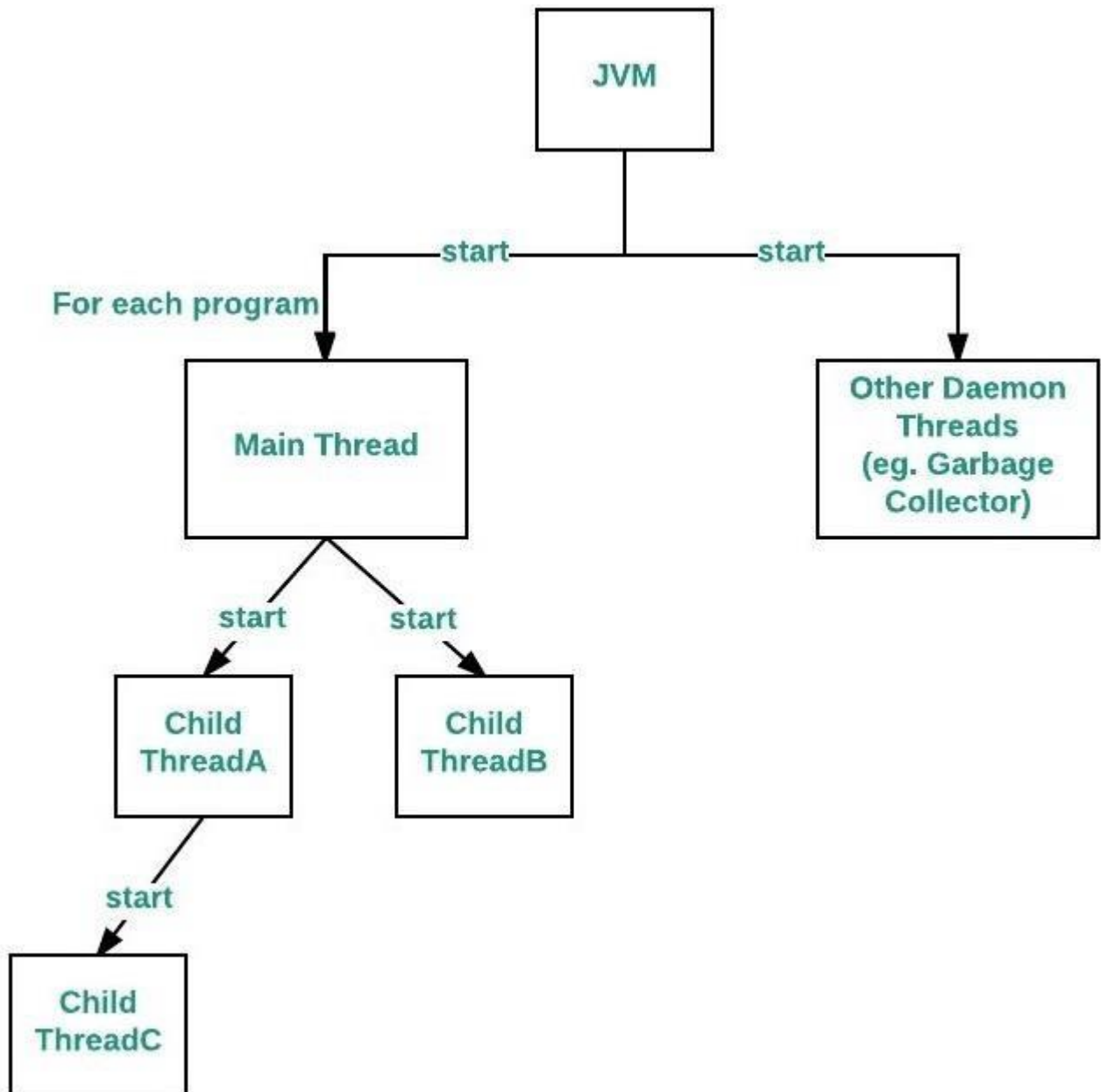
33)	void	<u>setContextClassLoader()</u>	It sets the context ClassLoader for the Thread.
34)	ClassLoader	<u>getContextClassLoader()</u>	It returns the context ClassLoader for the thread.
35)	static Thread.UncaughtExceptionHandler	<u>getDefaultUncaughtExceptionHandler()</u>	It returns the default handler invoked when a thread abruptly terminates due to an uncaught exception.
36)	static void	<u>setDefaultUncaughtExceptionHandler()</u>	It sets the default handler invoked when a thread abruptly terminates due to an uncaught exception

The Main Thread:

Java provides built-in support for multithreaded programming. A multi-threaded program contains two or more parts that can run concurrently. Each part of such a program is called a thread, and each thread defines a separate path of execution. When a Java program starts up, one thread begins running immediately. This is usually called the *main* thread of our program because it is the one that is executed when our program begins. There are certain properties associated with the main thread which are as follows:

- It is the thread from which other “child” threads will be spawned.
- Often, it must be the last thread to finish execution because it performs various shutdown actions

The flow diagram is as follows:



How to control Main thread

The main thread is created automatically when our program is started. To control it we must obtain a reference to it. This can be done by calling the method `currentThread()` which is present in Thread class. This method returns a reference to the thread on which it is called. The default priority of Main thread is 5 and for all remaining user threads priority will be inherited from parent to child.

Example

- Java

```
// Java program to control the Main Thread
// Importing required classes

import java.io.*;

import java.util.*;

// Class 1
// Main class extending thread class

public class Test extends Thread {

    // Main driver method

    public static void main(String[] args)
    {

        // Getting reference to Main thread

        Thread t = Thread.currentThread();

        // Getting name of Main thread

        System.out.println("Current thread: "

                            + t.getName());
```

```
// Changing the name of Main thread

t.setName("Geeks");

System.out.println("After name change: "

                    + t.getName());


// Getting priority of Main thread

System.out.println("Main thread priority: "

                    + t.getPriority());


// Setting priority of Main thread to MAX(10)

t.setPriority(MAX_PRIORITY);


// Print and display the main thread priority

System.out.println("Main thread new priority: "

                    + t.getPriority());


for (int i = 0; i < 5; i++) {

    System.out.println("Main thread");

}
```

```
// Main thread creating a child thread

Thread ct = new Thread() {

    // run() method of a thread

    public void run()

    {

        for (int i = 0; i < 5; i++) {

            System.out.println("Child thread");

        }

    }

};


// Getting priority of child thread
// which will be inherited from Main thread
// as it is created by Main thread

System.out.println("Child thread priority: "

    + ct.getPriority());


// Setting priority of Main thread to MIN(1)

ct.setPriority(MIN_PRIORITY);
```

```
        System.out.println("Child thread new priority: "
                            + ct.getPriority());

        // Starting child thread

        ct.start();

    }
}

// Class 2
// Helper class extending Thread class
// Child Thread class
class ChildThread extends Thread {

    @Override public void run()
    {

        for (int i = 0; i < 5; i++) {

            // Print statement whenever child thread is
            // called

            System.out.println("Child thread");
```

```
    }  
    }  
}
```

Output

Current thread: main

After name change: Geeks

Main thread priority: 5

Main thread new priority: 10

Main thread

Main thread

Main thread

Main thread

Main thread

Child thread priority: 10

Child thread new priority: 1

Child thread

Child thread

Child thread

Child thread

Child thread

Now let us discuss the relationship between the `main()` method and the main thread in Java. For each program, a Main thread is created by JVM(Java Virtual Machine). The “Main” thread first verifies the existence of the `main()` method, and then it initializes the class. Note that from JDK 6, `main()` method is mandatory in a standalone java application.

Deadlocking with use of Main Thread(only single thread)

We can create a deadlock by just using the Main thread, i.e. by just using a single thread.

Example

- Java

```
// Java program to demonstrate deadlock

// using Main thread


// Main class

public class GFG {


    // Main driver method

    public static void main(String[] args) {


        // Try block to check for exceptions

        try{


            // Print statement

            System.out.println("Entering into Deadlock");


            // Joining the current thread

            Thread.currentThread().join();


            // This statement will never execute

            System.out.println("This statement will never execute");
```



```

    }

    // Catch block to handle the exceptions

    catch (InterruptedException e) {

        // Display the exception along with line number

        // using printStackTrace() method

        e.printStackTrace();

    }

}

}

```

Output:

```

[mayanksolanki@Mayanks-MacBook-Air test % javac GFG.java
[mayanksolanki@Mayanks-MacBook-Air test % java GFG
Entering into Deadlock
>

```

Output explanation:

The statement “Thread.currentThread().join()”, will tell Main thread to wait for this thread(i.e. wait for itself) to die. Thus Main thread wait for itself to die, which is nothing but a deadlock.

Java Thread Model:

The Java language and its run-time system was designed keeping in mind about multithreading. The run-time system depend upon multithreading. Java provides asynchronous thread environment, this helps to increase the utilization of CPU. Multithreading is best in all cases in contrast with single-thread model. Single-thread system uses an approach of event loop with polling. According to this approach a

single thread in the system runs in an infinite loop. Polling the mechanism, that selects a single event from the event queue to choose what to do next. As the event is selected, then event loop forwards the control to the corresponding required event handler. Nothing else can be happened, until the event handler returns. Because of this CPU time is wasted. Here, only one part of the complete program is dominating the whole system, and preventing the system to execute or start any other process. In single-thread model one thread blocks all other threads until its execution completes. On other waiting or idle thread can start and acquire the resource which is not in use by the current thread. This causes the wastage of resources.

Java's multithreading provides benefit in this area by eliminating the loop and polling mechanism, one thread can be paused without stopping the other parts of the program. If any thread is paused or blocked, still other threads continue to run.

As the process has several states, similarly a thread exists in several states. A thread can be in the following states:

Ready to run (New): First time as soon as it gets CPU time.

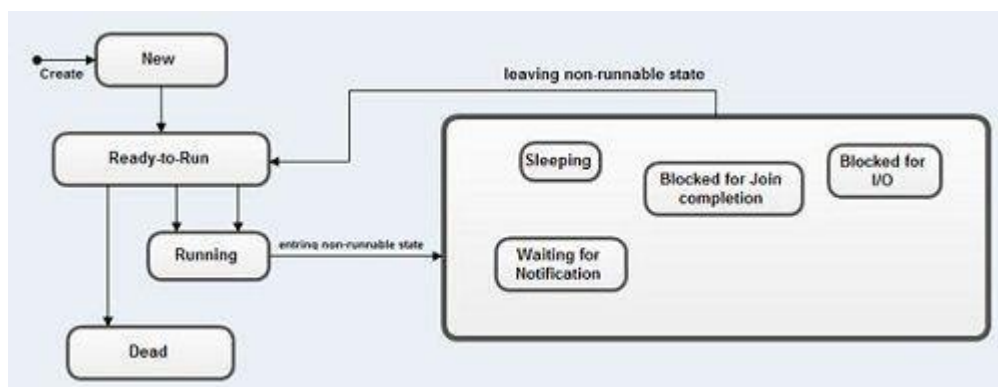
Running: Under execution.

Suspended: Temporarily not active or under execution.

Blocked: Waiting for resources.

Resumed: Suspended thread resumed, and start from where it left off.

Terminated: Halts the execution immediately and never resumes.



Java thread model can be defined in the following three sections:

We'll be covering the following topics in this tutorial:

- [Thread Priorities](#)
- [Synchronization](#)
- [Messaging](#)

Thread Priorities

Each thread has its own priority in Java. Thread priority is an absolute integer value. Thread priority decides only when a thread switches from one running thread to next, called *context switching*. Priority does increase the running time of the thread or gives faster execution.

Synchronization

Java supports an asynchronous multithreading, any number of thread can run simultaneously without disturbing other to access individual resources at different instant of time or shareable resources. But some time it may be possible that shareable resources are used by at least two threads or more than two threads, one has to write at the same time, or one has to write and other thread is in the middle of reading it. For such type of situations and circumstances Java implements synchronization model called *monitor*. The monitor was first defined by C.A.R. Hoare. You can consider the monitor as a box, in which only one thread can reside. As a thread enter in monitor, all other threads have to wait until that thread exits from the monitor. In such a way, a monitor protects the shareable resources used by it being manipulated by other waiting threads at the same instant of time. Java provides a simple methodology to implement synchronization.

Messaging

A program is a collection of more than one thread. Threads can communicate with each other. Java supports messaging between the threads with lost-cost. It provides methods to all objects for inter-thread communication. As a thread exits from synchronization state, it notifies all the waiting threads.

Thread Priorities:

Each thread has a priority. Priorities are represented by a number between 1 and 10. In most cases, the thread scheduler schedules the threads according to their priority (known as preemptive scheduling). But it is not guaranteed because it depends on JVM specification that which scheduling it chooses. Note that not only JVM a Java programmer can also assign the priorities of a thread explicitly in a Java program.

Setter & Getter Method of Thread Priority

Let's discuss the setter and getter method of the thread priority.

public final int getPriority(): The `java.lang.Thread.getPriority()` method returns the priority of the given thread.

public final void setPriority(int newPriority): The `java.lang.Thread.setPriority()` method updates or assign the priority of the thread to `newPriority`. The method throws `IllegalArgumentException` if the value `newPriority` goes out of the range, which is 1 (minimum) to 10 (maximum).

3 constants defined in Thread class:

1. `public static int MIN_PRIORITY`

2. public static int NORM_PRIORITY
3. public static int MAX_PRIORITY

Default priority of a thread is 5 (NORM_PRIORITY). The value of MIN_PRIORITY is 1 and the value of MAX_PRIORITY is 10.

Example of priority of a Thread:

FileName: ThreadPriorityExample.java

```
1. // Importing the required classes
2. import java.lang.*;
3.
4. public class ThreadPriorityExample extends Thread
5. {
6.
7. // Method 1
8. // Whenever the start() method is called by a thread
9. // the run() method is invoked
10. public void run()
11. {
12. // the print statement
13. System.out.println("Inside the run() method");
14. }
15.
16. // the main method
17. public static void main(String args[])
18. {
19. // Creating threads with the help of ThreadPriorityExample class
20. ThreadPriorityExample th1 = new ThreadPriorityExample();
21. ThreadPriorityExample th2 = new ThreadPriorityExample();
22. ThreadPriorityExample th3 = new ThreadPriorityExample();
23.
24. // We did not mention the priority of the thread.
25. // Therefore, the priorities of the thread is 5, the default value
26.
27. // 1st Thread
28. // Displaying the priority of the thread
```

```
29. // using the getPriority() method
30. System.out.println("Priority of the thread th1 is : " + th1.getPriority());
31.
32. // 2nd Thread
33. // Display the priority of the thread
34. System.out.println("Priority of the thread th2 is : " + th2.getPriority());
35.
36. // 3rd Thread
37. // // Display the priority of the thread
38. System.out.println("Priority of the thread th2 is : " + th2.getPriority());
39.
40. // Setting priorities of above threads by
41. // passing integer arguments
42. th1.setPriority(6);
43. th2.setPriority(3);
44. th3.setPriority(9);
45.
46. // 6
47. System.out.println("Priority of the thread th1 is : " + th1.getPriority());
48.
49. // 3
50. System.out.println("Priority of the thread th2 is : " + th2.getPriority());
51.
52. // 9
53. System.out.println("Priority of the thread th3 is : " + th3.getPriority());
54.
55. // Main thread
56.
57. // Displaying name of the currently executing thread
58. System.out.println("Currently Executing The Thread : " + Thread.currentThread().getName());
59.
60. System.out.println("Priority of the main thread is : " + Thread.currentThread().getPriority());
61.
62. // Priority of the main thread is 10 now
63. Thread.currentThread().setPriority(10);
64.
65. System.out.println("Priority of the main thread is : " + Thread.currentThread().getPriority());
```

66. }
67. }

Output:

```
Priority of the thread th1 is : 5  
Priority of the thread th2 is : 5  
Priority of the thread th2 is : 5  
Priority of the thread th1 is : 6  
Priority of the thread th2 is : 3  
Priority of the thread th3 is : 9  
Currently Executing The Thread : main  
Priority of the main thread is : 5  
Priority of the main thread is : 10
```

We know that a thread with high priority will get preference over lower priority threads when it comes to the execution of threads. However, there can be other scenarios where two threads can have the same priority. All of the processing, in order to look after the threads, is done by the Java thread scheduler. Refer to the following example to comprehend what will happen if two threads have the same priority.

FileName: ThreadPriorityExample1.java

```
1. // importing the java.lang package  
2. import java.lang.*;  
3.  
4. public class ThreadPriorityExample1 extends Thread  
5. {  
6.  
7. // Method 1  
8. // Whenever the start() method is called by a thread  
9. // the run() method is invoked  
10. public void run()  
11. {  
12. // the print statement  
13. System.out.println("Inside the run() method");  
14. }  
15.  
16.  
17. // the main method  
18. public static void main(String args[])  
19. {  
20.
```

```

21. // Now, priority of the main thread is set to 7
22. Thread.currentThread().setPriority(7);
23.
24. // the current thread is retrieved
25. // using the currentThread() method
26.
27. // displaying the main thread priority
28. // using the getPriority() method of the Thread class
29. System.out.println("Priority of the main thread is : " + Thread.currentThread().getPriority());
30.
31. // creating a thread by creating an object of the class ThreadPriorityExample1
32. ThreadPriorityExample1 th1 = new ThreadPriorityExample1();
33.
34. // th1 thread is the child of the main thread
35. // therefore, the th1 thread also gets the priority 7
36.
37. // Displaying the priority of the current thread
38. System.out.println("Priority of the thread th1 is : " + th1.getPriority());
39. }
40. }

```

Output:

```

Priority of the main thread is : 7
Priority of the thread th1 is : 7

```

Explanation: If there are two threads that have the same priority, then one can not predict which thread will get the chance to execute first. The execution then is dependent on the thread scheduler's algorithm (First Come First Serve, Round-Robin, etc.)

Example of IllegalArgumentException

We know that if the value of the parameter *newPriority* of the method *getPriority()* goes out of the range (1 to 10), then we get the *IllegalArgumentException*. Let's observe the same with the help of an example.

FileName: *IllegalArgumentException.java*

```

1. // importing the java.lang package
2. import java.lang.*;

```

```

3.
4. public class IllegalArgumentException extends Thread
5. {
6.
7. // the main method
8. public static void main(String args[])
9. {
10.
11. // Now, priority of the main thread is set to 17, which is greater than 10
12. Thread.currentThread().setPriority(17);
13.
14. // The current thread is retrieved
15. // using the currentThread() method
16.
17. // displaying the main thread priority
18. // using the getPriority() method of the Thread class
19. System.out.println("Priority of the main thread is : " + Thread.currentThread().getPriority());
20.
21. }
22. }

```

When we execute the above program, we get the following exception:

```

Exception in thread "main" java.lang.IllegalArgumentException
    at java.base/java.lang.Thread.setPriority(Thread.java:1141)
    at IllegalArgumentException.main(IllegalArgumentException.java:12)

```

Synchronization in java:

Synchronization in Java is the capability to control the access of multiple threads to any shared resource.

Java Synchronization is better option where we want to allow only one thread to access the shared resource.

Why use Synchronization?

The synchronization is mainly used to

1. To prevent thread interference.
2. To prevent consistency problem.

Types of Synchronization

There are two types of synchronization

1. Process Synchronization
2. Thread Synchronization

Here, we will discuss only thread synchronization.

Thread Synchronization

There are two types of thread synchronization mutual exclusive and inter-thread communication.

1. Mutual Exclusive
 1. Synchronized method.
 2. Synchronized block.
 3. Static synchronization.
2. Cooperation (Inter-thread communication in java)

Mutual Exclusive

Mutual Exclusive helps keep threads from interfering with one another while sharing data. It can be achieved by using the following three ways:

1. By Using Synchronized Method
2. By Using Synchronized Block
3. By Using Static Synchronization

Concept of Lock in Java

Synchronization is built around an internal entity known as the lock or monitor. Every object has a lock associated with it. By convention, a thread that needs consistent access to an object's fields has to acquire the object's lock before accessing them, and then release the lock when it's done with them.

From Java 5 the package `java.util.concurrent.locks` contains several lock implementations.

Understanding the problem without Synchronization

In this example, there is no synchronization, so output is inconsistent. Let's see the example:

TestSynchronization1.java

```
1. class Table{
2. void printTable(int n){//method not synchronized
3. for(int i=1;i<=5;i++){
4.     System.out.println(n*i);
5.     try{
6.         Thread.sleep(400);
7.     }catch(Exception e){System.out.println(e);}
8. }
9.
10. }
11. }
12.
13. class MyThread1 extends Thread{
14. Table t;
15. MyThread1(Table t){
16. this.t=t;
17. }
18. public void run(){
19. t.printTable(5);
20. }
21.
22. }
23. class MyThread2 extends Thread{
24. Table t;
25. MyThread2(Table t){
26. this.t=t;
27. }
```

```
28. public void run(){
29. t.printTable(100);
30. }
31. }
32.
33. class TestSynchronization1{
34. public static void main(String args[]){
35. Table obj = new Table();//only one object
36. MyThread1 t1=new MyThread1(obj);
37. MyThread2 t2=new MyThread2(obj);
38. t1.start();
39. t2.start();
40. }
41. }
```

Output:

```
5
100
10
200
15
300
20
400
25
500
```

Java Synchronized Method

If you declare any method as synchronized, it is known as synchronized method.

Synchronized method is used to lock an object for any shared resource.

When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

TestSynchronization2.java

```
1. //example of java synchronized method
2. class Table{
3.     synchronized void printTable(int n){//synchronized method
4.         for(int i=1;i<=5;i++){
```

```
5.    System.out.println(n*i);
6.    try{
7.        Thread.sleep(400);
8.    }catch(Exception e){System.out.println(e);}
9.    }
10.
11. }
12. }
13.
14. class MyThread1 extends Thread{
15. Table t;
16. MyThread1(Table t){
17.     this.t=t;
18. }
19. public void run(){
20. t.printTable(5);
21. }
22.
23. }
24. class MyThread2 extends Thread{
25. Table t;
26. MyThread2(Table t){
27.     this.t=t;
28. }
29. public void run(){
30. t.printTable(100);
31. }
32. }
33.
34. public class TestSynchronization2{
35. public static void main(String args[]){
36. Table obj = new Table();//only one object
37. MyThread1 t1=new MyThread1(obj);
38. MyThread2 t2=new MyThread2(obj);
39. t1.start();
40. t2.start();
41. }
```

42. }

Output:

```
5
10
15
20
25
100
200
300
400
500
```

Example of synchronized method by using anonymous class

In this program, we have created the two threads by using the anonymous class, so less coding is required.

TestSynchronization3.java

```
1. //Program of synchronized method by using anonymous class
2. class Table{
3.     synchronized void printTable(int n){//synchronized method
4.         for(int i=1;i<=5;i++){
5.             System.out.println(n*i);
6.             try{
7.                 Thread.sleep(400);
8.             }catch(Exception e){System.out.println(e);}
9.         }
10.
11. }
12. }
13.
14. public class TestSynchronization3{
15.     public static void main(String args[]){
16.         final Table obj = new Table();//only one object
17.
18.         Thread t1=new Thread(){
19.             public void run(){
```

```
20. obj.printTable(5);
21. }
22. };
23. Thread t2=new Thread(){
24. public void run(){
25. obj.printTable(100);
26. }
27. };
28.
29. t1.start();
30. t2.start();
31. }
32. }
```

Output:

```
5
10
15
20
25
100
200
300
400
500
```

Interthread Communication:

Inter-thread communication or **Co-operation** is all about allowing synchronized threads to communicate with each other.

Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed. It is implemented by following methods of **Object class**:

- wait()
- notify()
- notifyAll()

1) wait() method

The wait() method causes current thread to release the lock and wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.

The current thread must own this object's monitor, so it must be called from the synchronized method only otherwise it will throw exception.

Method				Description
public	final	void	wait()throws InterruptedException	It waits until object is notified.
public	final	void	wait(long timeout)throws InterruptedException	It waits for the specified amount of time.

2) notify() method

The notify() method wakes up a single thread that is waiting on this object's monitor. If any threads are waiting on this object, one of them is chosen to be awakened. The choice is arbitrary and occurs at the discretion of the implementation.

Syntax:

1. **public final void** notify()

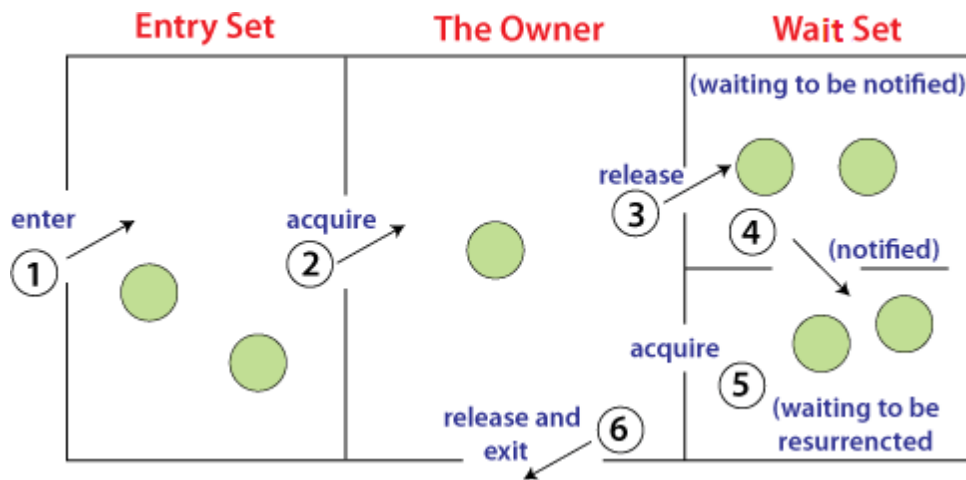
3) notifyAll() method

Wakes up all threads that are waiting on this object's monitor.

Syntax:

1. **public final void** notifyAll()

Understanding the process of inter-thread communication



The point to point explanation of the above diagram is as follows:

1. Threads enter to acquire lock.
2. Lock is acquired by on thread.
3. Now thread goes to waiting state if you call wait() method on the object. Otherwise it releases the lock and exits.
4. If you call notify() or notifyAll() method, thread moves to the notified state (runnable state).
5. Now thread is available to acquire lock.
6. After completion of the task, thread releases the lock and exits the monitor state of the object.

Why wait(), notify() and notifyAll() methods are defined in Object class not Thread class?

It is because they are related to lock and object has a lock.

Difference between wait and sleep?

Let's see the important differences between wait and sleep methods.

wait()	sleep()
The wait() method releases the lock.	The sleep() method doesn't release the lock.
It is a method of Object class	It is a method of Thread class

It is the non-static method	It is the static method
It should be notified by notify() or notifyAll() methods	After the specified amount of time, sleep is completed.

Example of Inter Thread Communication in Java

Let's see the simple example of inter thread communication.

Test.java

```

1. class Customer{
2.     int amount=10000;
3.
4.     synchronized void withdraw(int amount){
5.         System.out.println("going to withdraw...");
6.
7.         if(this.amount<amount){
8.             System.out.println("Less balance; waiting for deposit...");
9.             try{wait();}catch(Exception e){}
10.        }
11.        this.amount-=amount;
12.        System.out.println("withdraw completed...");
13.    }
14.
15.    synchronized void deposit(int amount){
16.        System.out.println("going to deposit...");
17.        this.amount+=amount;
18.        System.out.println("deposit completed... ");
19.        notify();
20.    }
21. }
22.
23. class Test{
24.     public static void main(String args[]){
25.         final Customer c=new Customer();
26.         new Thread(){
27.             public void run(){c.withdraw(15000);}

```

```
28. }.start();
29. new Thread(){
30. public void run(){c.deposit(10000);}
31. }.start();
32.
33. }}
```

Output:

```
going to withdraw...
Less balance; waiting for deposit...
going to deposit...
deposit completed...
withdraw completed
```

I/O in Java

I/O Basics:

Java I/O (Input and Output) is used *to process the input and produce the output*. Java uses the concept of a stream to make I/O operation fast. The java.io package contains all the classes required for input and output operations. We can perform **file handling in Java** by Java I/O API.

Stream

A stream is a sequence of data. In Java, a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.

In Java, 3 streams are created for us automatically. All these streams are attached with the console.

1) System.out: standard output stream

2) System.in: standard input stream

3) System.err: standard error stream

Let's see the code to print **output and an error** message to the console.

1. `System.out.println("simple message");`
2. `System.err.println("error message");`

Let's see the code to get **input** from console.

1. `int i=System.in.read();//returns ASCII code of 1st character`
2. `System.out.println((char)i);//will print the character`

OutputStream vs InputStream

The explanation of OutputStream and InputStream classes are given below:

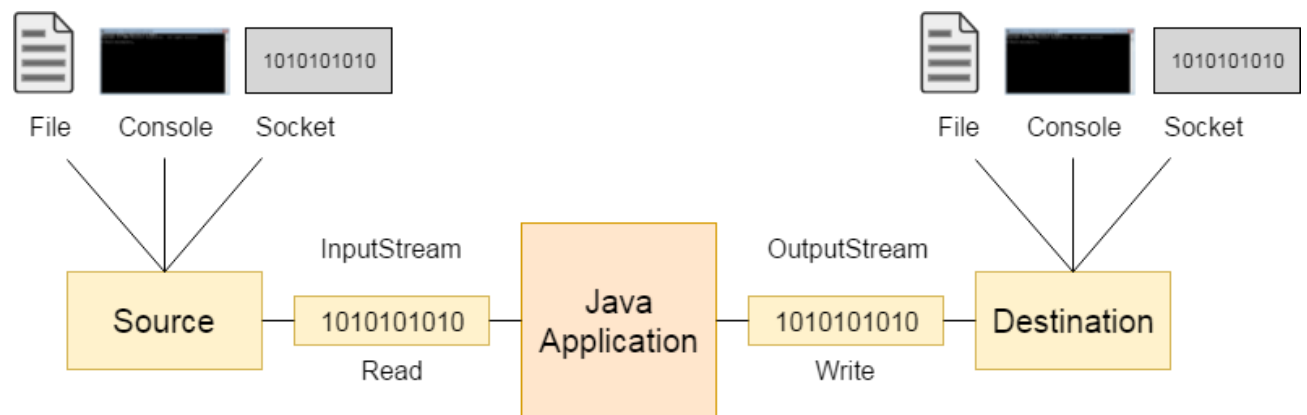
OutputStream

Java application uses an output stream to write data to a destination; it may be a file, an array, peripheral device or socket.

InputStream

Java application uses an input stream to read data from a source; it may be a file, an array, peripheral device or socket.

Let's understand the working of Java OutputStream and InputStream by the figure given below.



OutputStream class

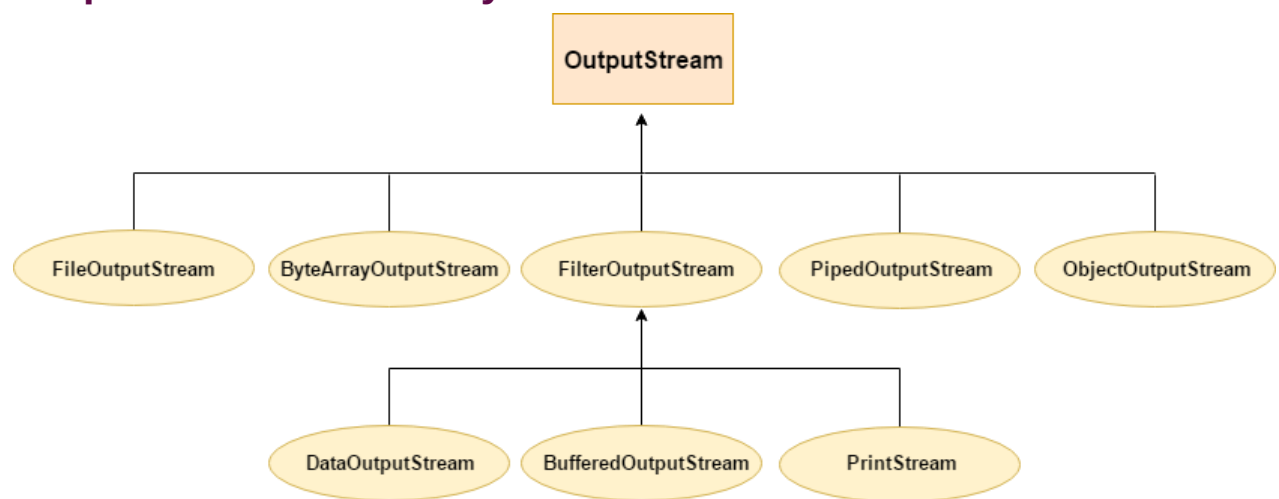
OutputStream class is an abstract class. It is the superclass of all classes representing an output stream of bytes. An output stream accepts output bytes and sends them to some sink.

Useful methods of OutputStream

Method	Description
1) public void write(int)throws IOException	is used to write a byte to the current output stream.
2) public void write(byte[])throws IOException	is used to write an array of byte to the current output stream.

3) public void flush()throws IOException	flushes the current output stream.
4) public void close()throws IOException	is used to close the current output stream.

OutputStream Hierarchy



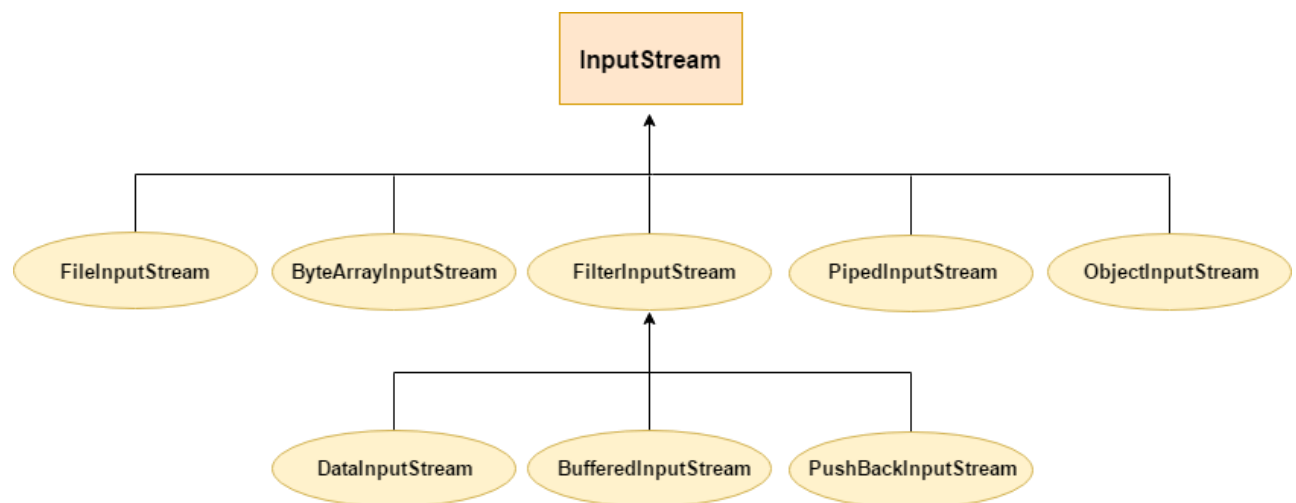
InputStream class

InputStream class is an abstract class. It is the superclass of all classes representing an input stream of bytes.

Useful methods of InputStream

Method	Description
1) public abstract int read()throws IOException	reads the next byte of data from the input stream. It returns -1 at the end of the file.
2) public int available()throws IOException	returns an estimate of the number of bytes that can be read from the current input stream.
3) public void close()throws IOException	is used to close the current input stream.

InputStream Hierarchy



The Predefined Streams:

Every language has some predefined streams for its users and Java is one of these languages. Three Java Predefined streams or standard streams are available in the `java.lang.System` class. These are as follows:

System.in	This is the standard stream for input . This stream is used for reading data for the program from the keyboard by default.
System.out	This is the standard stream for output . This stream is used for writing data from the program to an output device such as a monitor / console by default or to some specified file.
System.err	This is a standard stream for error . This is used to show an error message on the screen i.e. console by default for the users.

System.in is an object of **InputStream**. On the other hand, System.out and System.err are both an object of type **PrintStream**.

All these Java Predefined Streams are automatically initialized by Java's JVM (Java Virtual Machine) on startup. All these streams are byte streams but these are used to read and write character streams as well.

Let us have a close look at these streams one at a time.

Java Predefined Stream for Standard Input

System.in is the stream used for standard input in Java. This stream is an object of **InputStream** stream. In Java you need to wrap the System.in in the BufferedReader object in order to obtain a character based stream.

This is done with the following code:

```
BufferedReader br = new BufferedReader (new InputStreamReader (System.in) );
```

Here, InputStreamReader is the input stream you are creating and br is the character based stream that will be linked through System.in. Now, br object can be used for reading inputs from users.

In order to understand how this stream is used, let us have a look at an **example**.

Program to show use of System.in of Java Predefined Streams

```
import java.io.*;
class InputEg
{
    public static void main( String args[] ) throws IOException
    {
        String city;
        BufferedReader br = new BufferedReader ( new
InputStreamReader (System.in) );
        System.out.println ( "Where do you live?" );
        city = br.readLine();
        System.out.println ( "You have entered " + city + "
city" );
    }
}
```

Output:

```
D:\Programs>javac InputEg.java
D:\Programs>java InputEg
Where do you live?
Punjab
You have entered Punjab city
D:\Programs>
```

This stream is a bit complicated so it is used less often as inputs are either passed on command line or are passed via GUI depending on the applications. In many console applications and applets, the job of `System.in` is done by using `System.out` itself.

Java Predefined Stream for Standard Output

`System.out` is an object of the **PrintStream** stream. This stream is the stream used for standard output in Java. The output of this stream is directed to the console by default by the program. Have a look at the above example.

The statement *`System.out.println( "You have entered " + city + " city" );`* of the code will direct the output of the program on the console of the user.

Java Predefined Stream for Standard Error

`System.err` is the stream used for standard error in Java. This stream is an object of **PrintStream** stream.

System.err is similar to `System.out` but it is most commonly used inside the catch section of the try / catch block. A sample of the block is as follows:

```
try {

// Code for execution

}

catch ( Exception e) {

System.err.println ( "Error in code: " + e );
```


Reading From Console:

In Java, there are four different ways for reading input from the user in the command line environment(console).

1.Using Buffered Reader Class

This is the Java classical method to take input, Introduced in JDK1.0. This method is used by wrapping the System.in (standard input stream) in an InputStreamReader which is wrapped in a BufferedReader, we can read input from the user in the command line.

- The input is buffered for efficient reading.
- The wrapping code is hard to remember.

Implementation:

- Java

```
// Java program to demonstrate BufferedReader

import java.io.BufferedReader;

import java.io.IOException;

import java.io.InputStreamReader;

public class Test {

    public static void main(String[] args)

        throws IOException

    {

        // Enter data using BufferReader

        BufferedReader reader = new BufferedReader (

            new InputStreamReader(System.in));

        // Reading data using readLine
```

```
String name = reader.readLine();

// Printing the read line

System.out.println(name);

}

}
```

Input:

Geek

Output:

Auxiliary Space : $O(1)$

Geek

Note:

To read other types, we use functions like `Integer.parseInt()`, `Double.parseDouble()`. To read multiple values, we use `split()`.

2. Using Scanner Class

This is probably the most preferred method to take input. The main purpose of the `Scanner` class is to parse primitive types and strings using regular expressions, however, it is also can be used to read input from the user in the command line.

- Convenient methods for parsing primitives (`nextInt()`, `nextFloat()`, ...) from the tokenized input.
- Regular expressions can be used to find tokens.
- The reading methods are not synchronized

To see more differences, please see [this](#) article.

- **Java**

```
// Java program to demonstrate working of Scanner in Java

import java.util.Scanner;

class GetInputFromUser {

    public static void main(String args[])

    {

        // Using Scanner for Getting Input from User

        Scanner in = new Scanner(System.in);

        String s = in.nextLine();

        System.out.println("You entered string " + s);

        int a = in.nextInt();

        System.out.println("You entered integer " + a);

        float b = in.nextFloat();

        System.out.println("You entered float " + b);

    }

}
```

Input:

GeeksforGeeks

12

3.4

Output:

You entered string GeeksforGeeks

You entered integer 12

You entered float 3.4

3. Using Console Class

It has been becoming a preferred way for reading user's input from the command line. In addition, it can be used for reading password-like input without echoing the characters entered by the user; the format string syntax can also be used (like `System.out.printf()`).

Advantages:

- Reading password without echoing the entered characters.
- Reading methods are synchronized.
- Format string syntax can be used.
- Does not work in non-interactive environment (such as in an IDE).

• Java

```
// Java program to demonstrate working of System.console()

// Note that this program does not work on IDEs as

// System.console() may require console

public class Sample {
```

```
public static void main(String[] args)

{

    // Using Console to input data from user

    String name = System.console().readLine();


    System.out.println("You entered string " + name);

}

}
```

Input:

GeeksforGeeks

Output:

You entered string GeeksforGeeks

4. Using Command line argument

Most used user input for competitive coding. The command-line arguments are stored in the String format. The parseInt method of the Integer class converts string argument into Integer. Similarly, for float and others during execution. The usage of args[] comes into existence in this input form. The passing of information takes place during the program run. The command line is given to args[]. These programs have to be run on cmd.

Code:

- Java

[illegible]

Command Line Arguments:

```
javac GFG1.java  
java Main Hello World
```

Output:

The command line arguments are:

Hello

World

Writing to Console:

Writing Console Output in Java refers to writing the output of a Java program to the console or any particular file.

The methods used for streaming output are defined in the **PrintStream** class. The methods used for writing console output are `print()`, `println()` and `write()`.

Methods of Writing Console Output in Java-`print()` and `println()`

Both `print()` and `println()` methods are used to direct the output to the console. These methods are defined in the **PrintStream** class and are widely used. Both these methods are used with the help of the `System.out` stream.

The basic differences between `print()` and `println()` methods are as follows:

- **`print()`** method displays the string in the same line whereas **`println()`** method outputs a newline character after its execution.
- **`print()`** method is used for directing output to console only whereas **`println()`** method is used for directing output to not only console but other sources also.

Now, let us understand it in a better way with the help of some examples.

Example of `print()` method:

```
// Sample program to display numbers from 1-10 using print method
class printEg
{
    public static void main(String args[])
    {
        int a;
        for (a=1 ; a<=10 ; a++)
        {
            System.out.print(a);
        }
    }
}
```

Output:

```
D:\Programs>javac printEg.java
D:\Programs>java printEg
12345678910
D:\Programs>
```

Example of println() method:

Let us take the same example above and use println() method in place of print() method.

```
// Sample program to display numbers from 1-10 using println method
class printlnEg
{
    public static void main(String args[])
    {
        int a;
        for (a=1 ; a<=10 ; a++)
        {
            System.out.println(a);
        }
    }
}
```

Output:

```
D:\Programs>javac printlnEg.java
D:\Programs>java printlnEg
1
2
3
4
5
6
7
8
9
10
D:\Programs>
```

Observe the difference in the output.

Both these functions work best for output of strings as well. Let us have a look at another example.

Displaying strings using print() and println() methods

It is very easy to display strings using print() and println() methods. For displaying any particular string, the string to be displayed on screen is written within the double quotation marks (i.e. “ ”).

For example: `System.out.println(“Hello!! How are you?”);`

In addition, if you want to display the value of any particular variable used in the program, then you have to append the variable name along with the string with a plus (+) symbol.

`System.out.println("Name entered by you is: " + name);`



Example for displaying any **string** output:

```
// Sample program to display sum of numbers from 1-10
class Eg1
{
    public static void main(String args[])
    {
        int a, total = 0;
        for (a=1 ; a<=10 ; a++)
        {
            total = total + a;
        }
        System.out.println("Sum of first 10 numbers is: " +
total);
    }
}
```

Output:

```
D:\Programs>javac Eg1.java
D:\Programs>java Eg1
Sum of first 10 numbers is: 55
D:\Programs>
```

The write() method

Alternatively, you can make use of the write() method for directing the output of your program to the console. The easiest **syntax** of the write() method is:

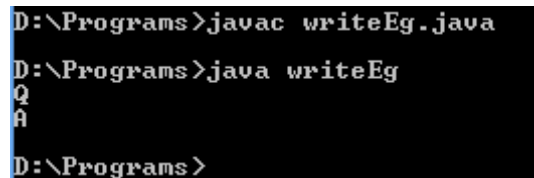
```
void write(int b);
```

Where b is an integer of low order eight bits.

Let us have a look at a simple **example**.

```
class writeEg
{
    public static void main(String args[])
    {
        int a, b;
        a = 'Q';
        b = 65;
        System.out.write(a);
        System.out.write('\n');
        System.out.write(b);
        System.out.write('\n');
    }
}
```

Output:



```
D:\Programs>javac writeEg.java
D:\Programs>java writeEg
Q
A
D:\Programs>
```

Reading and Writing Files in Java:

Java FileWriter and FileReader classes are used to write and read data from text files (they are [Character Stream](#) classes). It is recommended **not** to use the FileInputStream and FileOutputStream classes if you have to read and write any textual information as these are Byte stream classes.

FileWriter

FileWriter is useful to create a file writing characters into it.

- This class inherits from the OutputStream class.
- The constructors of this class assume that the default character encoding and the default byte-buffer size are acceptable. To specify these values yourself, construct an OutputStreamWriter on a FileOutputStream.

- **FileWriter** is meant for writing streams of characters. For writing streams of raw bytes, consider using a **FileOutputStream**.
- **FileWriter** creates the output file if it is not present already.

Constructors:

- **FileWriter(File file)** – Constructs a **FileWriter** object given a **File** object.
- **FileWriter (File file, boolean append)** – constructs a **FileWriter** object given a **File** object.
- **FileWriter (FileDescriptor fd)** – constructs a **FileWriter** object associated with a file descriptor.
- **FileWriter (String fileName)** – constructs a **FileWriter** object given a file name.
- **FileWriter (String fileName, Boolean append)** – Constructs a **FileWriter** object given a file name with a **Boolean** indicating whether or not to append the data written.

Methods:

- **public void write (int c) throws IOException** – Writes a single character.
- **public void write (char [] str) throws IOException** – Writes an array of characters.
- **public void write(String str)throws IOException** – Writes a string.
- **public void write(String str, int off, int len)throws IOException** – Writes a portion of a string. Here off is offset from which to start writing characters and len is the number of characters to write.
- **public void flush() throws IOException** flushes the stream
- **public void close() throws IOException** flushes the stream first and then closes the writer.

Reading and writing take place character by character, which increases the number of I/O operations and affects the performance of the system.**BufferedWriter** can be used along with **FileWriter** to improve the speed of execution.

The following program depicts how to create a text file using **FileWriter**

• Java

```
// Creating a text File using FileWriter

import java.io.FileWriter;

import java.io.IOException;

class CreateFile

{
```

```

public static void main(String[] args) throws IOException
{
    // Accept a string
    String str = "File Handling in Java using "+
        " FileWriter and FileReader";

    // attach a file to FileWriter
    FileWriter fw=new FileWriter("output.txt");

    // read character wise from string and write
    // into FileWriter
    for (int i = 0; i < str.length(); i++)
        fw.write(str.charAt(i));

    System.out.println("Writing successful");
    //close the file
    fw.close();
}
}

```

FileReader

FileReader is useful to read data in the form of characters from a 'text' file.

- This class inherited from the InputStreamReader Class.

- The constructors of this class assume that the default character encoding and the default byte-buffer size are appropriate. To specify these values yourself, construct an `InputStreamReader` on a `FileInputStream`.
- `FileReader` is meant for reading streams of characters. For reading streams of raw bytes, consider using a `FileInputStream`.

Constructors:

- **`FileReader(File file)`** – Creates a `FileReader` , given the `File` to read from
- **`FileReader(FileDescriptor fd)`** – Creates a new `FileReader` , given the `FileDescriptor` to read from
- **`FileReader(String fileName)`** – Creates a new `FileReader` , given the name of the file to read from

Methods:

- **`public int read () throws IOException`** – Reads a single character. This method will block until a character is available, an I/O error occurs, or the end of the stream is reached.
- **`public int read(char[] cbuf) throws IOException`** – Reads characters into an array. This method will block until some input is available, an I/O error occurs, or the end of the stream is reached.
- **`public abstract int read(char[] buff, int off, int len) throws IOException`** – Reads characters into a portion of an array. This method will block until some input is available, an I/O error occurs, or the end of the stream is reached.

Parameters:

`cbuf` - Destination buffer

`off` - Offset at which to start storing characters

`len` - Maximum number of characters to read

- **`public void close() throws IOException`** closes the reader.
- **`public long skip(long n) throws IOException`** – Skips characters. This method will block until some characters are available, an I/O error occurs, or the end of the stream is reached.

Parameters:

`n` - The number of characters to skip

The following program depicts how to read from the 'text' file using `FileReader`

Java

```
// Reading data from a file using FileReader

import java.io.FileNotFoundException;
```

```
import java.io.FileReader;

import java.io.IOException;

class ReadFile

{

    public static void main(String[] args) throws IOException

    {

        // variable declaration

        int ch;

        // check if File exists or not

        FileReader fr=null;

        try

        {

            fr = new FileReader("text");

        }

        catch (FileNotFoundException fe)

        {

            System.out.println("File not found");

        }

        // read from FileReader till the end of file
```

```

        while ((ch=fr.read())!=-1)

            System.out.print((char)ch);


        // close the file

        fr.close();

    }

}

```

The Transient and Volatile Modifiers:

Volatile:

The Volatile keyword is used to mark the JVM and thread to read its value from primary memory and not utilize cached value present in the thread stack. It is used in concurrent programming in java.

- Java

```

class GFG extends Thread {

    // using volatile

    volatile boolean working = true;


    // if non-volatile it will

    // sleep in main and

```

```
// runtime error will coocur

public void run()

{
    long count = 0;

    while (working) {

        count++;

    }

    System.out.println("Thread terminated."
                        + count);
}


// Driver code
public static void main(String[] args)

    throws InterruptedException

{
    GFG test = new GFG();

    test.start();

    Thread.sleep(100);

    System.out.println("After Sleeping in Main");

    test.working = false;
```



```

        test.join();

        System.out.println("Working set to "

                           + test.working);

    }

}

```

Output:

```

"C:\Program Files\Java\jdk1.8.0_
After Sleeping in Main
Thread terminated.302022069
Working set to false

```

Volatile Code Output

Transient:

The Transient keyword is used with the instance variable to eliminate it from the serialization process. During serialization, the value of the transient field or variable is not saved.

• Java

```

import java.io.*;

class Test implements Serializable {

    // Making Accesskey transient for security

    transient String accessKey;

    // Making age transient as age can be

```

```
// calculated from Date of Birth

// and current date.

transient int age;


// serialize other fields

String name, address;

public Test (String accessKey, int age,
             String name, String address)

{
    this.accessKey = accessKey;

    this.age = age;

    this.name = name;

    this.address = address;

}

}

public class GFG {

    public static void main (String[] args)

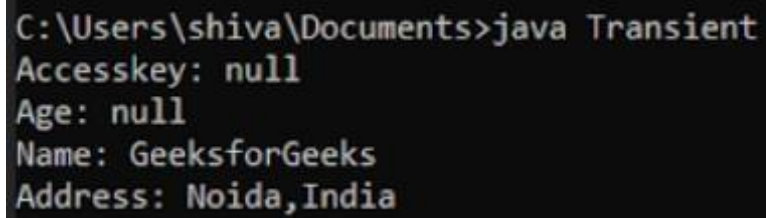
        throws Exception

    {

        ObjectInputStream in
```

```
        = new ObjectInputStream(  
            (new FileInputStream(  
                "login_details.txt")));  
  
Test obj = (Test)in.readObject();  
  
/* Transient variable will be shown  
null due to security reasons.*/  
  
System.out.println("Accesskey: "  
                    + obj.accessKey);  
  
System.out.println("Age: "  
                    + obj.age);  
  
System.out.println("Name: "  
                    + obj.name);  
  
System.out.println("Address: "  
                    + obj.address);  
    }  
}
```

Output:



```
C:\Users\shiva\Documents>java Transient  
Accesskey: null  
Age: null  
Name: GeeksforGeeks  
Address: Noida,India
```

Transient Code Output

The following table describes the differences:

Transient	Volatile
The Transient marked variable prevents it from being serialized.	The Volatile marked variable follows happen-before relationship on visibility in multithreaded java program which reduces the risk of memory consistency errors.
It gives control and flexibility over to exclude some object methods from serialization process.	It prevents JVM from doing re-ordering which could compromise synchronization.
During deserialization they are initialized with a default value.	They are not initialized with a default value.
It cannot be used with the static keyword as static variable do not belong to individual instance. During serialization, only object's current state is concerned.	It can be used with the static keyword.
It cannot be used with the final keyword. Although JVM doesn't complain about it but during deserialization one will face problem of reinitializing a variable.	It can be used with the final keyword.

Using Instance of Native Methods:

The native keyword in Java is applied to a method to indicate that the method is implemented in native code using JNI (Java Native Interface). The native keyword is a modifier that is applicable **only for methods**, and we can't apply it anywhere else. The methods which are implemented in C, C++ are called native methods or foreign methods.

The native modifier indicates that a method is implemented in platform-dependent code, often seen in C language. Native modifier indicates that a method is implemented in platform-dependent code, often in C.

Main objectives of the native keyword

- To improve the performance of the system.
- To achieve machine level/memory level communication.
- To use already existing legacy non-java code.

Conclusion: *Java application can call code is written in C, C++, or assembler.*

Steps for the creation of native methods are as follows:

1. Write java code
2. Compile the java code.
3. Create C header(.h file)
4. Create C stubs file (using tool: Java HEdge)
5. Write C code
6. Create a shared code library (DLL)
7. Run application

Implementation: Native keyword in Java

Let us first take random Java code that contains the native method and later we will be compiling it. We are done with the above two steps. For steps 3 and 4 we will be using the existing .exe known as java HEdge” in order to create C header and C stub file.

Now we will insert(write) our C code(or use) and later using DLL, we will be creating objects of the same inside our application(Main example1A) and later calling the native methods thereby within the java program.

Example 1-A: Application

- Java

```
// Java Program to Illustrate Native Keyword
```

```
// Inside DLL named: NameOfDLLFile
```

```
// Main class
```

```
// NativeDemo
```

```
class GFG {

    // Method 1
    public static void main(String[] args)
    {

        int var;

        // Here we will not be having body of this method
        // in our java code here
        NameOfDLLFile obj = new NameOfDLLFile();

        obj.var = null;

        System.out.println("Before native method: var = "
                           + var);

        obj.test();

        System.out.println("After native method: var = "
```

```

        + var);

    }

    // Native method
    public native void test()

    {

        static

        {

            // We will be loading body from DLL file

            // It has to be present in DLL file

            System.loadLibrary("NameOfDLLFile");

            // Above C code is loaded in the JVM

        }

    }
}

```

For above program C code that is shared in DLL is as follows:

Example 1-B: Support to above example

- C++

```
// C++ Program to Be Shared In DLL to Illustrate  
// Native Method in Java
```

```
// Importing required libraries
```

```
#include <iostream>
```

```
using namespace std;
```

```
// Method 2
```

```
// Native
```

```
void test(int var)
```

```
{
```

```
    cout << var;
```

```
}
```

```
// Method 1
```

```
// Main driver method
```

```
int main()
```

```
{
```

```
    test(10);
```



```
    return 1;

}
```

Note: DLL is named as can be perceived from program 1A: **NameOfDLLFile**
Output:

Before native method: var = null

After native method: var = 10

Do remember that there are certain **important points about native keywords**, which are as follows:

- For native methods, implementation is already available in old languages like C, C++ and we are not responsible for providing an implementation. Hence native method declaration should end with ; (semi-colon).
- We can't declare a native method as abstract.
- We can't declare a native method as strictfp because there is no guarantee that old languages (C, C++) follow IEEE 754 standard. Hence native strictfp combination is an illegal combination for methods.
- The main advantage of native keyword is performance improvement, but the main disadvantage of native keyword is that it breaks the platform-independent nature of java.

Note: Do go through strictfp keyword of java as is one of the concept which even very good java developer is unaware of.

In this section, we explain how to declare a native method in Java and how to generate the corresponding C/C++ function prototype.

Syntax: Declaring Native Methods

```
private native String getLine(String prompt);
```

Syntax: From the Native Language Side

```
javah -jni Prompt
```

```
JNIEXPORT jstring JNICALL Java_Prompt_getLine(JNIEnv *, jobject, jstring);
```

STRING AND CHARACTERS

Fundamentals of Characters and Strings:

In [Java](#), string is basically an object that represents sequence of char values. An [array](#) of characters works same as Java string. For example:

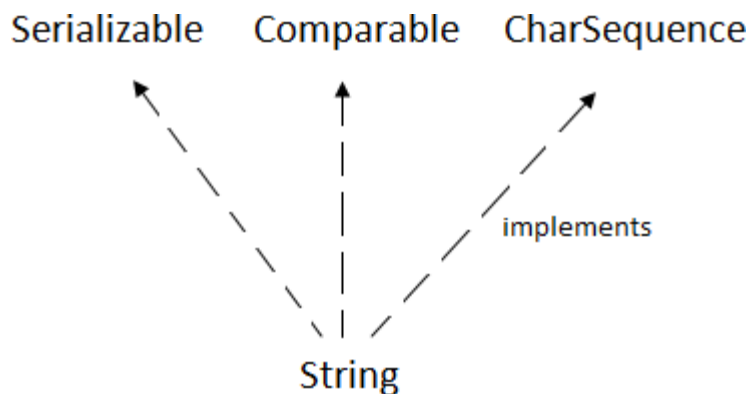
1. `char[] ch={'j','a','v','a','t','p','o','i','n','t'};`
2. `String s=new String(ch);`

is same as:

1. `String s="javatpoint";`

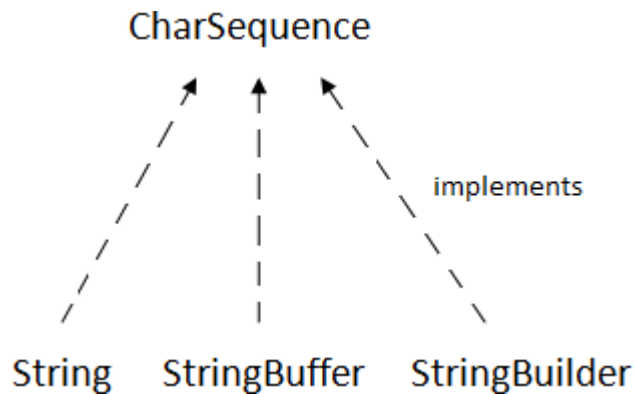
Java String class provides a lot of methods to perform operations on strings such as `compare()`, `concat()`, `equals()`, `split()`, `length()`, `replace()`, `compareTo()`, `intern()`, `substring()` etc.

The `java.lang.String` class implements *Serializable*, *Comparable* and *CharSequence* [interfaces](#).



CharSequence Interface

The `CharSequence` interface is used to represent the sequence of characters. `String`, [StringBuffer](#) and [StringBuilder](#) classes implement it. It means, we can create strings in Java by using these three classes.



The Java String is immutable which means it cannot be changed. Whenever we change any string, a new instance is created. For mutable strings, you can use `StringBuffer` and `StringBuilder` classes.

We will discuss immutable string later. Let's first understand what String in Java is and how to create the String object.

What is String in Java?

Generally, String is a sequence of characters. But in Java, string is an object that represents a sequence of characters. The `java.lang.String` class is used to create a string object.

How to create a string object?

There are two ways to create String object:

1. By string literal
2. By new keyword

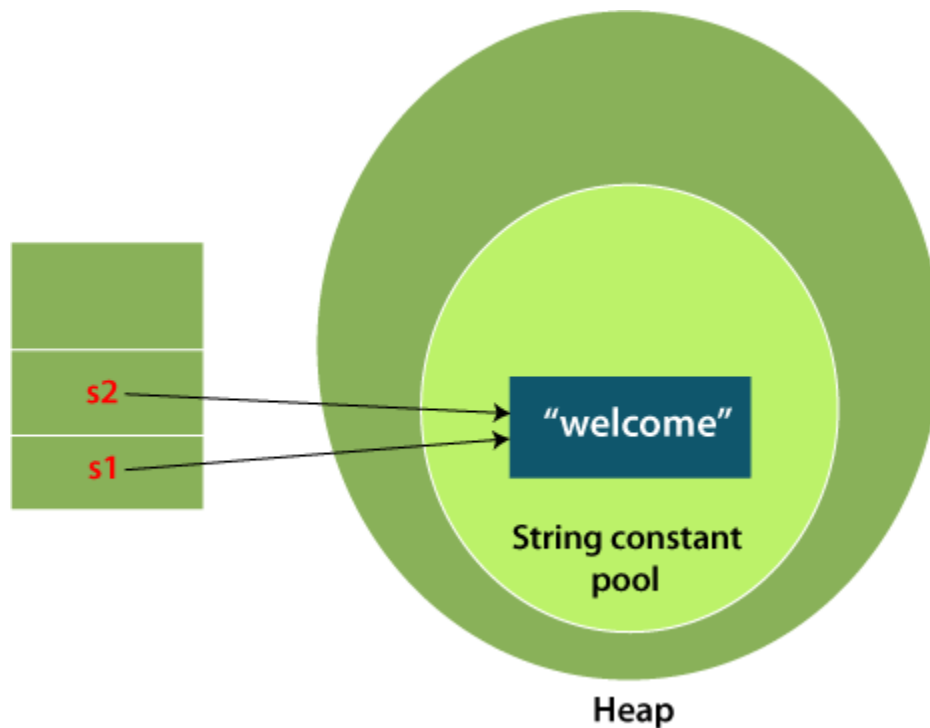
1) String Literal

Java String literal is created by using double quotes. For Example:

1. `String s="welcome";`

Each time you create a string literal, the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new string instance is created and placed in the pool. For example:

1. String s1="Welcome";
2. String s2="Welcome";//It doesn't create a new instance



In the above example, only one object will be created. Firstly, JVM will not find any string object with the value "Welcome" in string constant pool that is why it will create a new object. After that it will find the string with the value "Welcome" in the pool, it will not create a new object but will return the reference to the same instance.

Note: String objects are stored in a special memory area known as the "string constant pool".

Why Java uses the concept of String literal?

To make Java more memory efficient (because no new objects are created if it exists already in the string constant pool).

2) By new keyword

1. String s=**new** String("Welcome");//creates two objects and one reference variable
In such case, JVM will create a new string object in normal (non-pool) heap memory, and the literal "Welcome" will be placed in the string constant pool. The variable s will refer to the object in a heap (non-pool).

Java String Example

StringExample.java

1. **public class** StringExample{
2. **public static void** main(String args[]){
3. String s1="java";//creating string by Java string literal
4. **char** ch[]={**'s','t','r','i','n','g','s'**};
5. String s2=**new** String(ch);//converting char array to string
6. String s3=**new** String("example");//creating Java string by new keyword
7. System.out.println(s1);
8. System.out.println(s2);
9. System.out.println(s3);
10. }}

[Test it Now](#)

Output:

```
java
strings
example
```

The above code, converts a **char** array into a **String** object. And displays the String objects **s1**, **s2**, and **s3** on console using **println()** method.

Java String class methods

The java.lang.String class provides many useful methods to perform operations on sequence of char values.

No.	Method	Description
1	char charAt(int index)	It returns char value for the particular index
2	int length()	It returns string length
3	static String format(String format, Object... args)	It returns a formatted string.

4	<u>static String format(Locale l, String format, Object... args)</u>	It returns formatted string with given locale.
5	<u>String substring(int beginIndex)</u>	It returns substring for given begin index.
6	<u>String substring(int beginIndex, int endIndex)</u>	It returns substring for given begin index and end index.
7	<u>boolean contains(CharSequence s)</u>	It returns true or false after matching the sequence of char value.
8	<u>static String join(CharSequence delimiter, CharSequence... elements)</u>	It returns a joined string.
9	<u>static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)</u>	It returns a joined string.
10	<u>boolean equals(Object another)</u>	It checks the equality of string with the given object.
11	<u>boolean isEmpty()</u>	It checks if string is empty.
12	<u>String concat(String str)</u>	It concatenates the specified string.
13	<u>String replace(char old, char new)</u>	It replaces all occurrences of the specified char value.
14	<u>String replace(CharSequence old, CharSequence new)</u>	It replaces all occurrences of the specified CharSequence.
15	<u>static String equalsIgnoreCase(String another)</u>	It compares another string. It doesn't check case.
16	<u>String[] split(String regex)</u>	It returns a split string matching regex.

17	<u>String[] split(String regex, int limit)</u>	It returns a split string matching regex and limit.
18	<u>String intern()</u>	It returns an interned string.
19	<u>int indexOf(int ch)</u>	It returns the specified char value index.
20	<u>int indexOf(int ch, int fromIndex)</u>	It returns the specified char value index starting with given index.
21	<u>int indexOf(String substring)</u>	It returns the specified substring index.
22	<u>int indexOf(String substring, int fromIndex)</u>	It returns the specified substring index starting with given index.
23	<u>String toLowerCase()</u>	It returns a string in lowercase.
24	<u>String toLowerCase(Locale l)</u>	It returns a string in lowercase using specified locale.
25	<u>String toUpperCase()</u>	It returns a string in uppercase.
26	<u>String toUpperCase(Locale l)</u>	It returns a string in uppercase using specified locale.
27	<u>String trim()</u>	It removes beginning and ending spaces of this string.
28	<u>static String valueOf(int value)</u>	It converts given type into string. It is an overloaded method.

The String Class:

String is a sequence of characters. In java, objects of String are immutable which means a constant and cannot be changed once created.

Creating a String

There are two ways to create string in Java:

- **String literal**
String s = "GeeksforGeeks";
- **Using new keyword**
String s = new String ("GeeksforGeeks");

Constructors

1. **String(byte[] byte_arr)** - Construct a new String by decoding the *byte array*. It uses the platform's default character set for decoding.

Example:

2. byte[] b_arr = {71, 101, 101, 107, 115};
3. String s_byte = new String(b_arr); //Geeks
4. **String(byte[] byte_arr, Charset char_set)** - Construct a new String by decoding the *byte array*. It uses the *char_set* for decoding.

Example:

5. byte[] b_arr = {71, 101, 101, 107, 115};
6. Charset cs = Charset.defaultCharset();
7. String s_byte_char = new String(b_arr, cs); //Geeks
8. **String(byte[] byte_arr, String char_set_name)** - Construct a new String by decoding the *byte array*. It uses the *char_set_name* for decoding. It looks similar to the above constructs and they appear before similar functions but it takes the *String(which contains char_set_name)* as parameter while the above constructor takes *CharSet*.

Example:

9. byte[] b_arr = {71, 101, 101, 107, 115};
10. String s = new String(b_arr, "US-ASCII"); //Geeks
11. **String(byte[] byte_arr, int start_index, int length)** - Construct a new string from the *bytes array* depending on the *start_index(Starting location)* and *length(number of characters from starting location)*.

Example:

12. byte[] b_arr = {71, 101, 101, 107, 115};
13. String s = new String(b_arr, 1, 3); // eek
14. **String(byte[] byte_arr, int start_index, int length, Charset char_set)** - Construct a new string from the *bytes array* depending on the *start_index(Starting location)* and *length(number of characters from starting location)*. Uses *char_set* for decoding.

Example:

15. byte[] b_arr = {71, 101, 101, 107, 115};
16. Charset cs = Charset.defaultCharset();
17. String s = new String(b_arr, 1, 3, cs); // eek

18. **String(byte[] byte_arr, int start_index, int length, String char_set_name)** - Construct a new string from the *bytes array* depending on the *start_index*(Starting location) and *length*(number of characters from starting location). Uses *char_set_name* for decoding.

Example:

- ```
19. byte[] b_arr = {71, 101, 101, 107, 115};
20. String s = new String(b_arr, 1, 4, "US-ASCII"); // eeks
```

21. **String(char[] char\_arr)** - Allocates a new String from the given *Character array*

**Example:**

- ```
22. char char_arr[] = {'G', 'e', 'e', 'k', 's'};
23. String s = new String(char_arr); //Geeks
```

24. **String(char[] char_array, int start_index, int count)** - Allocates a String from a given *character array* but choose *count* characters from the *start_index*.

Example:

- ```
25. char char_arr[] = {'G', 'e', 'e', 'k', 's'};
26. String s = new String(char_arr, 1, 3); //eek
```

27. **String(int[] uni\_code\_points, int offset, int count)** - Allocates a String from a *uni\_code\_array* but choose *count* characters from the *start\_index*.

**Example:**

- ```
28. int[] uni_code = {71, 101, 101, 107, 115};
29. String s = new String(uni_code, 1, 3); //eek
```

30. **String(StringBuffer s_buffer)** - Allocates a new string from the string in *s_buffer*

Example:

- ```
31. StringBuffer s_buffer = new StringBuffer("Geeks");
32. String s = new String(s_buffer); //Geeks
```

33. **String(StringBuilder s\_builder)** - Allocates a new string from the string in *s\_builder*

**Example:**

- ```
34. StringBuilder s_builder = new StringBuilder("Geeks");
35. String s = new String(s_builder); //Geeks
```

String Methods

1. **int length():** Returns the number of characters in the String.
"GeeksforGeeks".length(); // returns 13
2. **Char charAt(int i):** Returns the character at *ith* index.
"GeeksforGeeks".charAt(3); // returns 'k'

3. String substring (int i): Return the substring from the i^{th} index character to end.
`"GeeksforGeeks".substring(3); // returns "ksforGeeks"`
4. String substring (int i, int j): Returns the substring from i to j-1 index.
`"GeeksforGeeks".substring(2, 5); // returns "eks"`
5. String concat(String str): Concatenates specified string to the end of this string.
6. `String s1 = "Geeks";`
7. `String s2 = "forGeeks";`
8. `String output = s1.concat(s2); // returns "GeeksforGeeks"`
9. int indexOf (String s): Returns the index within the string of the first occurrence of the specified string.
10. `String s = "Learn Share Learn";`
11. `int output = s.indexOf("Share"); // returns 6`
12. int indexOf (String s, int i): Returns the index within the string of the first occurrence of the specified string, starting at the specified index.
13. `String s = "Learn Share Learn";`
14. `int output = s.indexOf("ea",3); // returns 13`
15. int lastIndexOf(String s): Returns the index within the string of the last occurrence of the specified string.
16. `String s = "Learn Share Learn";`
17. `int output = s.lastIndexOf("a"); // returns 14`
18. **boolean equals(Object otherObj)**: Compares this string to the specified object.
19. `Boolean out = "Geeks".equals("Geeks"); // returns true`
20. `Boolean out = "Geeks".equals("geeks"); // returns false`
21. boolean equalsIgnoreCase (String anotherString): Compares string to another string, ignoring case considerations.
22. `Boolean out= "Geeks".equalsIgnoreCase("Geeks"); // returns true`
`Boolean out = "Geeks".equalsIgnoreCase("geeks"); // returns true`
23. int compareTo(String anotherString): Compares two string lexicographically.
24. `int out = s1.compareTo(s2); // where s1 and s2 are`
25. `// strings to be compared`
- 26.
27. This returns difference s1-s2. If :

```
28.    out < 0  // s1 comes before s2
29.    out = 0  // s1 and s2 are equal.
```

```
30.    out > 0  // s1 comes after s2.
```

31. **int compareToIgnoreCase(String anotherString):** Compares two string lexicographically, ignoring case considerations.

```
32.    int out = s1.compareToIgnoreCase(s2);
33.    // where s1 and s2 are
34.    // strings to be compared
```

```
35.
```

```
36.    This returns difference s1-s2. If :
37.    out < 0  // s1 comes before s2
38.    out = 0  // s1 and s2 are equal.
39.    out > 0  // s1 comes after s2.
```

Note- In this case, it will not consider case of a letter (it will ignore whether it is uppercase or lowercase).

40. **String toLowerCase():** Converts all the characters in the String to lower case.

```
41.    String word1 = "HeLLo";
42.    String word3 = word1.toLowerCase(); // returns "hello"
```

43. **String toUpperCase():** Converts all the characters in the String to upper case.

```
44.    String word1 = "HeLLo";
45.    String word2 = word1.toUpperCase(); // returns "HELLO"
```

46. **String trim():** Returns the copy of the String, by removing whitespaces at both ends. It does not affect whitespaces in the middle.

```
47.    String word1 = " Learn Share Learn ";
48.    String word2 = word1.trim(); // returns "Learn Share Learn"
```

49. **String replace (char oldChar, char newChar):** Returns new string by replacing all occurrences of *oldChar* with *newChar*.

```
50.    String s1 = "feeksforfeeks";
51.    String s2 = "feeksforfeeks".replace('f' , 'g'); // returns
    "geeksgorgeeks"
```

Note:- s1 is still feeksforfeeks and s2 is geeksgorgeeks

Program to illustrate all string methods:

```
// Java code to illustrate different constructors and methods
// String class.

import java.io.*;

import java.util.*;

class Test
{
    public static void main (String[] args)
    {
        String s= "GeeksforGeeks";

        // or String s= new String ("GeeksforGeeks");

        // Returns the number of characters in the String.

        System.out.println("String length = " + s.length());

        // Returns the character at ith index.

        System.out.println("Character at 3rd position = "

                            + s.charAt(3));

        // Return the substring from the ith index character
```

```
// to end of string

System.out.println("Substring " + s.substring(3));


// Returns the substring from i to j-1 index.

System.out.println("Substring = " + s.substring(2,5));


// Concatenates string2 to the end of string1.

String s1 = "Geeks";

String s2 = "forGeeks";

System.out.println("Concatenated string = " +

                    s1.concat(s2));


// Returns the index within the string
// of the first occurrence of the specified string.

String s4 = "Learn Share Learn";

System.out.println("Index of Share " +

                    s4.indexOf("Share"));


// Returns the index within the string of the
// first occurrence of the specified string,
// starting at the specified index.
```

```
System.out.println("Index of a  = " +
                    s4.indexOf('a',3));

// Checking equality of Strings

Boolean out = "Geeks".equals("geeks");

System.out.println("Checking Equality " + out);

out = "Geeks".equals("Geeks");

System.out.println("Checking Equality " + out);


out = "Geeks".equalsIgnoreCase("gEeks ");

System.out.println("Checking Equality " + out);


//If ASCII difference is zero then the two strings are similar

int out1 = s1.compareTo(s2);

System.out.println("the difference between ASCII value is="+out1);

// Converting cases

String word1 = "GeeKyMe";

System.out.println("Changing to lower Case " +
                    word1.toLowerCase());


// Converting cases
```

```
String word2 = "GeekyME";

System.out.println("Changing to UPPER Case " +

                    word2.toUpperCase());

// Trimming the word

String word4 = " Learn Share Learn ";

System.out.println("Trim the word " + word4.trim());

// Replacing characters

String str1 = "feeksforfeeks";

System.out.println("Original String " + str1);

String str2 = "feeksforfeeks".replace('f' , 'g') ;

System.out.println("Replaced f with g -> " + str2);

}

}
```

Output :

String length = 13

Character at 3rd position = k

Substring ksforGeeks

Substring = eks

Concatenated string = GeeksforGeeks

Index of Share 6

Index of a = 8

Checking Equality false

Checking Equality true
Checking Equality false
the difference between ASCII value is=-31
Changing to lower Case geekyme
Changing to UPPER Case GEEKYME
Trim the word Learn Share Learn
Original String feeksforfeeks
Replaced f with g -> geeksgorgeeks

String Operation:

String Concatenation:

Concatenation is joining of two or more strings.

Have a look at the below picture-



We have two strings str1 = "Rock" and str2 = "Star"

If we add up these two strings, we should have a result as str3= "RockStar".

Check the below code snippet, and it explains the two methods to perform string concatenation.

First is using “**concat**” method of String class and second is using arithmetic “+” operator. Both results in the same output



```
public class Sample_String{  
  
    public static void main(String[] args){  
  
        //String Concatenation  
  
        String str1 = "Rock";  
  
        String str2 = "Star";  
  
        //Method 1 : Using concat  
  
        String str3 = str1.concat(str2);  
  
        System.out.println(str3);  
  
        //Method 2 : Using "+" operator  
  
        String str4 = str1 + str2;  
  
        System.out.println(str4);  
  
    }  
  
}
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

This code is editable. Click Run to Compile + Execute

Run



Important Java string methods:



Hey!! You turned me into a Rockstar in the last topic.

There are quite some more methods in the String class which are very helpful too.

Let's check them out in this topic.

Let's ask the Java String class a few questions and see if it can answer them:

String "Length" Method

How will you determine the length of given String? I have provided a method called as "length". Use it against the String you need to find the length.



```
public class Sample_String{
```

```
    public static void main(String[] args){ //Our sample string for this tutorial
```

```
        String str_Sample = "RockStar";
```

1

2

3

4

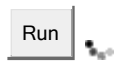
```
//Length of a String
```

5

```
System.out.println("Length of String: " + str_Sample.length());}}
```

6

This code is editable. Click Run to Compile + Execute



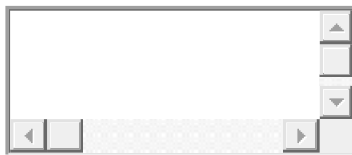
Expected Output:

Length of String: 8

String “indexOf” Method

If I know the length, how would I find which character is in which position? In short, how will I find the index of a character?

You answered yourself, buddy, there is an “indexOf” method that will help you determine the location of a specific character that you specify.



1

```
public class Sample_String{
```

2

```
    public static void main(String[] args){//Character at position
```

3

```
String str_Sample = "RockStar";
```

4

```
System.out.println("Character at position 5: " + str_Sample.charAt(5));
```

5

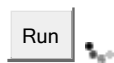
```
//Index of a given character
```

6

```
System.out.println("Index of character 'S': " + str_Sample.indexOf('S'));
```

7

This code is editable. Click Run to Compile + Execute



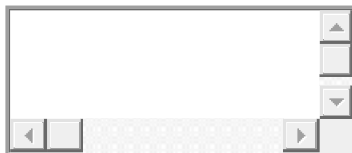
Expected Output:

```
Character at position 5: t  
Index of character 'S': 4
```

String “charAt” Method

Similar to the above question, given the index, how do I know the character at that location?

Simple one again!! Use the “charAt” method and provide the index whose character you need to find.



```
public class Sample_String{  
  
    public static void main(String[] args){//Character at position  
  
String str_Sample = "RockStar";  
  
System.out.println("Character at position 5: " + str_Sample.charAt(5));}}
```

1
2
3
4
5

This code is editable. Click Run to Compile + Execute



Expected Output:

```
Character at position 5: t
```

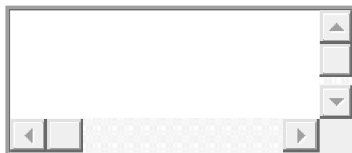
String “CompareTo” Method

Do I want to check if the String that was generated by some method is equal to something that I want to verify with? How do I compare two Strings?

Use the method “compareTo” and specify the String that you would like to compare.

Use “compareToIgnoreCase” in case you don’t want the result to be case sensitive.

The result will have the value 0 if the argument string is equal to this string; a value less than 0 if this string is lexicographically less than the string argument; and a value greater than 0 if this string is lexicographically greater than the string argument.



```
public class Sample_String{  
  
    public static void main(String[] args){//Compare to a String  
  
String str_Sample = "RockStar";  
  
    System.out.println("Compare To 'ROCKSTAR': " + str_Sample.compareTo("rockstar"));  
  
    //Compare to - Ignore case  
  
    System.out.println("Compare To 'ROCKSTAR' - Case Ignored: " +  
str_Sample.compareToIgnoreCase("ROCKSTAR"));}}
```

1
2
3
4
5
6
7

This code is editable. Click Run to Compile + Execute



Expected Output:

Compare To 'ROCKSTAR': -32

Compare To 'ROCKSTAR' - Case Ignored: 0

String “Contain” Method

I partially know what the string should have contained, how do I confirm if the String contains a sequence of characters I specify? Use the method “contains” and specify the characters you need to check.

Returns **true** if and only if this string contains the specified sequence of char values.

```
public class Sample_String{
    public static void main(String[] args){ //Check if String contains a
sequence
String str_Sample = "RockStar";
    System.out.println("Contains sequence 'tar': " +
str_Sample.contains("tar"));} }
```

Expected Output:

Contains sequence 'tar': true

String “endsWith” Method

How do I confirm if a String ends with a particular suffix? Again you answered it. Use the “endsWith” method and specify the suffix in the arguments.

Returns **true** if the character sequence represented by the argument is a suffix of the character sequence represented by this object.

```
public class Sample_String{
    public static void main(String[] args){ //Check if ends with a
particular sequence
String str_Sample = "RockStar";
    System.out.println("EndsWith character 'r': " +
str_Sample.endsWith("r"));} }
```

Expected Output:

EndsWith character 'r': true

String “replaceAll” & “replaceFirst” Method

I want to modify my String at several places and replace several parts of the String?

Java String Replace, replaceAll and replaceFirst methods. You can specify the part of the String you want to replace and the replacement String in the arguments.



```
public class Sample_String{  
    public static void main(String[] args){//Replace Rock with the word Duke  
  
    String str_Sample = "RockStar";  
  
    System.out.println("Replace 'Rock' with 'Duke': " + str_Sample.replace("Rock", "Duke"));}}
```

1
2
3
4
5

This code is editable. Click Run to Compile + Execute

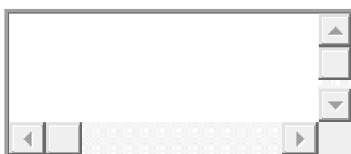


Expected Output:

Replace 'Rock' with 'Duke': DukeStar

String Java “tolowercase” & Java “touppercase” Method

I want my entire String to be shown in lower case or Uppercase?
Just use the “toLowerCase()” or “ToUpperCase()” methods against
the Strings that need to be converted.



```
public class Sample_String{  
  
    public static void main(String[] args){//Convert to LowerCase
```

1
2

```
String str_Sample = "RockStar";
```

3

```
System.out.println("Convert to LowerCase: " + str_Sample.toLowerCase());
```

4

```
//Convert to UpperCase
```

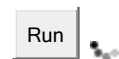
5

```
System.out.println("Convert to UpperCase: " + str_Sample.toUpperCase());}}
```

6

7

This code is editable. Click Run to Compile + Execute



Expected Output:

```
Convert to LowerCase: rockstar  
Convert to UpperCase: ROCKSTAR
```

Important Points to Note:

- **String is a Final class**; i.e once created the value cannot be altered. Thus String objects are called immutable.
- The Java Virtual Machine(JVM) creates a memory location especially for Strings called **String Constant Pool**. That's why String can be initialized without 'new' keyword.
- String class falls under **java.lang.String hierarchy**. But there is no need to import this class. Java platform provides them automatically.
- **String reference can be overridden but that does not delete the content**; i.e., if

```
String h1 = "hello";
```

```
h1 = "hello"+"world";
```

then "hello" String does not get deleted. It just loses its handle.

- **Multiple references** can be used for same String but it will **occur in the same place**; i.e., if


```
String h1 = "hello";
```

```
String h2 = "hello";
```

```
String h3 = "hello";
```

then only one pool for String "hello" is created in the memory with 3 references-h1,h2,h3

- If a **number is quoted in " "** then it **becomes a string**, not a number anymore. That means if

```
String S1 ="The number is: "+ "123"+"456";
```

```
System.out.println(S1);
```

then it will print: The number is: 123456

If the initialization is like this:

```
String S1 = "The number is: "+(123+456);
```

```
System.out.println(S1);
```

then it will print: The number is:579

Data Conversion Using Value of () Methods:

The **java string valueOf()** method converts different types of values into string. By the help of string valueOf() method, you can convert int to string, long to string, boolean to string, character to string, float to string, double to string, object to string and char array to string.

Internal implementation

1. **public static** String valueOf(Object obj) {
2. **return** (obj == **null**) ? "**null**" : obj.toString();
3. }

Signature

The signature or syntax of string valueOf() method is given below:

1. **public static** String valueOf(**boolean** b)
2. **public static** String valueOf(**char** c)
3. **public static** String valueOf(**char**[] c)
4. **public static** String valueOf(**int** i)
5. **public static** String valueOf(**long** l)
6. **public static** String valueOf(**float** f)
7. **public static** String valueOf(**double** d)
8. **public static** String valueOf(Object o)

Returns

string representation of given value

Java String valueOf() method example

1. **public class** StringValueOfExample{
2. **public static void** main(String args[]){
3. **int** value=30;
4. String s1=String.valueOf(value);
5. System.out.println(s1+10);//concatenating string with 10
6. }}

Test it Now

Output:

```
3010
```

Java String valueOf(boolean bol) Method Example

This is a boolean version of overloaded valueOf() method. It takes boolean value and returns a string. Let's see an example.

1. **public class** StringValueOfExample2 {
2. **public static void** main(String[] args) {
3. **// Boolean to String**
4. **boolean** bol = **true**;
5. **boolean** bol2 = **false**;
6. String s1 = String.valueOf(bol);

```
7.     String s2 = String.valueOf(bol2);
8.     System.out.println(s1);
9.     System.out.println(s2);
10.  }
11. }
```

Test it Now

Output:

```
true
false
```

Java String valueOf(char ch) Method Example

This is a char version of overloaded valueOf() method. It takes char value and returns a string. Let's see an example.

```
1.  public class StringValueOfExample3 {
2.      public static void main(String[] args) {
3.          // char to String
4.          char ch1 = 'A';
5.          char ch2 = 'B';
6.          String s1 = String.valueOf(ch1);
7.          String s2 = String.valueOf(ch2);
8.          System.out.println(s1);
9.          System.out.println(s2);
10.     }
11. }
```

Test it Now

Output:

```
A
B
```

Java String valueOf(float f) and valueOf(double d)

This is a float version of overloaded valueOf() method. It takes float value and returns a string. Let's see an example.

```
1.  public class StringValueOfExample4 {
```

```
2.  public static void main(String[] args) {
3.      // Float and Double to String
4.      float f = 10.05f;
5.      double d = 10.02;
6.      String s1 = String.valueOf(f);
7.      String s2 = String.valueOf(d);
8.      System.out.println(s1);
9.      System.out.println(s2);
10. }
11. }
```

Test it Now

Output:

```
10.05
10.02
```

Java String valueOf() Complete Examples

Let's see an example where we are converting all primitives and objects into strings.

```
1.  public class StringValueOfExample5 {
2.      public static void main(String[] args) {
3.          boolean b1=true;
4.          byte b2=11;
5.          short sh = 12;
6.          int i = 13;
7.          long l = 14L;
8.          float f = 15.5f;
9.          double d = 16.5d;
10.         char chr[]={ 'j','a','v','a' };
11.         StringValueOfExample5 obj=new StringValueOfExample5();
12.         String s1 = String.valueOf(b1);
13.         String s2 = String.valueOf(b2);
14.         String s3 = String.valueOf(sh);
15.         String s4 = String.valueOf(i);
16.         String s5 = String.valueOf(l);
17.         String s6 = String.valueOf(f);
18.         String s7 = String.valueOf(d);
```

```
19.    String s8 = String.valueOf(chr);
20.    String s9 = String.valueOf(obj);
21.    System.out.println(s1);
22.    System.out.println(s2);
23.    System.out.println(s3);
24.    System.out.println(s4);
25.    System.out.println(s5);
26.    System.out.println(s6);
27.    System.out.println(s7);
28.    System.out.println(s8);
29.    System.out.println(s9);
30. }
31. }
```

Test it Now

Output:

```
true
11
12
13
14
15.5
16.5
java
StringValueOfExample5@2a139a55
```

String Buffer Class and Methods:

Java StringBuffer class is used to create mutable (modifiable) String objects. The StringBuffer class in Java is the same as String class except it is mutable i.e. it can be changed.

Note: Java StringBuffer class is thread-safe i.e. multiple threads cannot access it simultaneously. So it is safe and will result in an order.

Important Constructors of StringBuffer Class

Constructor	Description
StringBuffer()	It creates an empty String buffer with the initial capacity of 16.

StringBuffer(String str)	It creates a String buffer with the specified string..
StringBuffer(int capacity)	It creates an empty String buffer with the specified capacity as length.

Important methods of StringBuffer class

Modifier and Type	Method	Description
public synchronized StringBuffer	append(String s)	It is used to append the specified string with this string. The append() method is overloaded like append(char), append(boolean), append(int), append(float), append(double) etc.
public synchronized StringBuffer	insert(int offset, String s)	It is used to insert the specified string with this string at the specified position. The insert() method is overloaded like insert(int, char), insert(int, boolean), insert(int, int), insert(int, float), insert(int, double) etc.
public synchronized StringBuffer	replace(int startIndex, int endIndex, String str)	It is used to replace the string from specified startIndex and endIndex.
public synchronized StringBuffer	delete(int startIndex, int endIndex)	It is used to delete the string from specified startIndex and endIndex.
public synchronized StringBuffer	reverse()	is used to reverse the string.
public int	capacity()	It is used to return the current capacity.
public void	ensureCapacity(int minimumCapacity)	It is used to ensure the capacity at least equal to the given minimum.

public char	charAt(int index)	It is used to return the character at the specified position.
public int	length()	It is used to return the length of the string i.e. total number of characters.
public String	substring(int beginIndex)	It is used to return the substring from the specified beginIndex.
public String	substring(int beginIndex, int endIndex)	It is used to return the substring from the specified beginIndex and endIndex.

What is a mutable String?

A String that can be modified or changed is known as mutable String. StringBuffer and StringBuilder classes are used for creating mutable strings.

1) StringBuffer Class append() Method

The append() method concatenates the given argument with this String.

StringBufferExample.java

1. **class** StringBufferExample{
2. **public static void** main(String args[]){
3. StringBuffer sb=**new** StringBuffer("Hello ");
4. sb.append("Java");//now original string is changed
5. System.out.println(sb);//prints Hello Java
6. }
7. }

Output:

```
Hello Java
```

2) StringBuffer insert() Method

The insert() method inserts the given String with this string at the given position.

StringBufferExample2.java

```
1. class StringBufferExample2{
2. public static void main(String args[]){
3. StringBuffer sb=new StringBuffer("Hello ");
4. sb.insert(1,"Java");//now original string is changed
5. System.out.println(sb);//prints HJavaello
6. }
7. }
```

Output:

```
HJavaello
```

3) StringBuffer replace() Method

The replace() method replaces the given String from the specified beginIndex and endIndex.

StringBufferExample3.java

```
1. class StringBufferExample3{
2. public static void main(String args[]){
3. StringBuffer sb=new StringBuffer("Hello");
4. sb.replace(1,3,"Java");
5. System.out.println(sb);//prints HJavallo
6. }
7. }
```

Output:

```
HJavallo
```

4) StringBuffer delete() Method

The delete () method of the StringBuffer class deletes the String from the specifiedbeginIndex to endIndex.

StringBufferExample4.java

```
1. class StringBufferExample4{
2. public static void main(String args[]){
3. StringBuffer sb=new StringBuffer("Hello");
4. sb.delete(1,3);
5. System.out.println(sb);//prints Hlo
```



```
6. }  
7. }
```

Output:

```
Hlo
```

5) StringBuffer reverse() Method

The reverse() method of the StringBuffer class reverses the current String.

StringBufferExample5.java

```
1. class StringBufferExample5{  
2. public static void main(String args[]){  
3. StringBuffer sb=new StringBuffer("Hello");  
4. sb.reverse();  
5. System.out.println(sb);//prints olleH  
6. }  
7. }
```

Output:

```
olleH
```

6) StringBuffer capacity() Method

The capacity() method of the StringBuffer class returns the current capacity of the buffer. The default capacity of the buffer is 16. If the number of character increases from its current capacity, it increases the capacity by $(oldcapacity * 2) + 2$. For example if your current capacity is 16, it will be $(16 * 2) + 2 = 34$.

StringBufferExample6.java

```
1. class StringBufferExample6{  
2. public static void main(String args[]){  
3. StringBuffer sb=new StringBuffer();  
4. System.out.println(sb.capacity());//default 16  
5. sb.append("Hello");  
6. System.out.println(sb.capacity());//now 16  
7. sb.append("java is my favourite language");  
8. System.out.println(sb.capacity());//now  $(16 * 2) + 2 = 34$  i.e  $(oldcapacity * 2) + 2$   
9. }
```

10. }

Output:

```
16
16
34
```

7) StringBuffer ensureCapacity() method

The ensureCapacity() method of the StringBuffer class ensures that the given capacity is the minimum to the current capacity. If it is greater than the current capacity, it increases the capacity by $(oldcapacity * 2) + 2$. For example if your current capacity is 16, it will be $(16 * 2) + 2 = 34$.

StringBufferExample7.java

```
1. class StringBufferExample7{
2.     public static void main(String args[]){
3.         StringBuffer sb=new StringBuffer();
4.         System.out.println(sb.capacity());//default 16
5.         sb.append("Hello");
6.         System.out.println(sb.capacity());//now 16
7.         sb.append("java is my favourite language");
8.         System.out.println(sb.capacity());//now (16*2)+2=34 i.e (oldcapacity*2)+2
9.         sb.ensureCapacity(10);//now no change
10.        System.out.println(sb.capacity());//now 34
11.        sb.ensureCapacity(50);//now (34*2)+2
12.        System.out.println(sb.capacity());//now 70
13.    }
14. }
```

Output:

```
16
16
34
34
70
```